BOOT FORCE — THE BUILD BIBLE

Complete Reference Document v1.0

A Comprehensive Blueprint for Building the Boot Force Club Hub Platform

TABLE OF CONTENTS

1. Project Identity & Strategy
2. System Architecture
3. Tech Stack — Complete Dependency Map
4. Data Model — Every Table, Every Relationship
5. Authentication & Security Architecture
6. Design System
7. Complete Route Map & Page Inventory
8. Feature Specifications — Every Module
9. Seed Data Strategy
10. Deployment & Environment Configuration
11. Git Workflow
12. The Demo Script
13. Phase Breakdown — 8 Phases
14. Risk Register & Mitigation
15. Post-Hackathon Roadmap

SECTION 1: PROJECT IDENTITY & STRATEGY

1.1 What Boot Force Is

Boot Force is the official digital hub for a technical club at CMR University. It is a single web application that replaces fragmented communication across WhatsApp groups, Google Forms, Google Drives, and social media pages. It centralizes member management, event hosting, challenge systems, project showcases, media galleries, and performance tracking into one cohesive platform.

It is NOT a multi-tenant platform. It is NOT a club-spawning system. It serves one club — Boot Force — with one URL, one database, one deployment.

1.2 The Problem It Solves

Pain Point	How Boot Force Solves It
Club info is fragmented	One platform — team, events, projects, gallery, all discoverable from one URL
Team members have no public visibility	Curated team section with photos, roles, skills, and social links
Students miss events	Events page with countdown timers, categories, and in-app notifications
No competitive ecosystem	Challenge system with submissions, grading, and real-time leaderboards
Projects are never showcased	Project portfolio with tech stacks, live demos, and community appreciation
Event photos are buried	Organized gallery with albums, masonry layout, and lightbox viewer

1.3 Target Users

Visitor (Guest): Anyone who opens the URL without logging in. They can browse every public page — homepage, team, events, challenges, projects, gallery, leaderboard, blog. They cannot interact (register, submit, like). When they attempt an interactive action, they are prompted to sign in. No login wall. No role selection gate.

Student (Member): A registered and authenticated user. They can do everything a visitor can, plus: register for events, submit challenge solutions, like projects, view their personal dashboard, earn badges, track streaks, write blog posts, download certificates, and receive in-app notifications.

Admin: A member with elevated privileges. They can do everything a member can, plus: full CRUD on events, challenges, team members, projects, gallery albums. They can review and grade submissions, manage users (promote, ban), publish announcements, view platform analytics, and see audit logs. Multiple admins are supported. The first admin is seeded in the database. Subsequent admins are promoted by existing admins.

1.4 Hackathon Scoring Strategy

Category	Weight	Our Strategy
UI/UX Design	25%	Maximum effort. Cinematic aesthetic, animations, micro-interactions, mobile-first. This is where we win or lose.
Innovation & Features	20%	The 3D/2D Live Club Pulse network visualization. Real-time leaderboard updates. On-demand certificate generation.
Functionality	20%	Core flows must work end-to-end without breaking. No dead links, no broken forms, no error screens visible to judges.
Club-Specific Modules	15%	Team section, gallery, events timeline, projects showcase — all present and populated with realistic seed data.
Presentation & Pitching	10%	3-minute demo script rehearsed and flawless. Pre-seeded data. Single login flow during demo.
Deployment	5%	Live on Vercel with HTTPS. Fast load time (<3 seconds on 4G).
Code Quality	5%	Clean modular structure, meaningful commit history, documented README.

Strategic rule: Every hour spent must be evaluated against this scoring table. If a feature doesn't improve our score in one of the top 3 categories, it gets built last.

SECTION 2: SYSTEM ARCHITECTURE

2.1 High-Level Architecture

The application follows a three-layer architecture:

Layer 1 — Client (Browser)
Next.js renders pages. Server Components handle initial data fetching and HTML generation on the server. Client Components handle interactivity, real-time subscriptions, and form state in the browser. The Supabase browser client manages real-time WebSocket connections.

Layer 2 — Server (Next.js on Vercel)
Next.js API routes and Server Actions handle mutations, file uploads, and any operation requiring the Supabase service role key. Middleware handles session refresh on every request. ISR (Incremental Static Regeneration) caches public pages and revalidates at defined intervals.

Layer 3 — Database & Services (Supabase)
PostgreSQL database with Row Level Security. Supabase Auth for authentication. Supabase Storage for files. Supabase Realtime for WebSocket subscriptions.

All three layers communicate via HTTPS. The Supabase anon key is public (safe for browser use with RLS). The service role key is server-only (used in Server Actions and API routes, never exposed to the client).

2.2 Data Fetching Architecture

There are exactly four methods of fetching or mutating data. Every page uses one or more of these. No exceptions.

Method 1 — Server Component Direct Fetch
Used for: All public pages (homepage, team, events listing, projects, gallery, leaderboard, blog).
How: The Server Component creates a Supabase server client (reads cookies for auth context), queries the database directly, and returns rendered HTML. No client-side JavaScript is shipped for the data fetch itself.
When: On initial page load and ISR revalidation.

Method 2 — Server Actions
Used for: All mutations (form submissions, registrations, likes, admin CRUD, grading, file uploads, profile updates).
How: Functions marked with `"use server"`. Called from Client Components. Execute on the server with full

access to the service role key if needed. Return typed responses: `{ success: boolean, data?: T, error?: string }`.
When: On user action (button click, form submit).

Method 3 — Client-Side Supabase SDK with TanStack Query
Used for: Any Client Component that needs data after initial render — specifically pages that receive server-fetched data as `initialData` and then manage updates, mutations, and caching client-side.
How: TanStack Query wraps Supabase client queries. `initialData` and `initialDataUpdatedAt` are passed from Server Components to avoid double-fetching. `staleTime` is set to match the ISR revalidation window.
When: Client-side navigation, background refetches, after mutations.

Method 4 — API Routes
Used for: Webhook endpoints, any external service callbacks, operations that don't fit the Server Action model.
How: Standard Next.js `route.ts` files in the app directory.
When: Rarely. Most operations use Server Actions. API routes are reserved for edge cases.

2.3 Server-to-Client Data Handoff Convention

This is a binding convention across the entire application:

1. Server Component fetches data and captures `dataUpdatedAt = Date.now()`
2. Server Component passes both `data` and `dataUpdatedAt` as props to the Client Component
3. Client Component initializes TanStack Query with `initialData: data` and `initialDataUpdatedAt: dataUpdatedAt`
4. `staleTime` is set per query (matching the ISR revalidation window for that page)
5. On client-side navigation (back/forward, link clicks), TanStack Query compares timestamps. If server data is newer, it replaces the cache. If the cache is newer (due to a mutation), the cache wins.
6. Serialization rule: All data passed from Server to Client Components must be plain objects. No Date objects, no undefined values, no class instances. Dates are ISO strings, parsed on the client.

Exception: Real-time components (leaderboard, notifications, announcement ticker) do NOT use `initialData`. They fetch on mount via TanStack Query, then Supabase Realtime pushes updates. A brief skeleton on first load is acceptable for these components.

2.4 State Management Strategy

Concern	Tool	Scope
Auth state (current user, role)	React Context	Root layout provider. Single source of truth. Updated by Supabase auth listener.
Server state (lists, entities, queries)	TanStack Query	Per-component or per-page. Handles caching, refetching, background updates, optimistic mutations.
Real-time state (leaderboard, notifications, announcements)	Supabase Realtime subscriptions → TanStack Query cache	Dedicated custom hooks. Callbacks write to TanStack Query cache via `queryClient.setQueryData()`.
Form state	React Hook Form	Per-form. Not global. Paired with Zod for validation.
UI state (modals, tabs, toggles)	React `useState`	Local to the component. No global store.
Global lightweight state	Zustand	NOT pre-installed. Added only if prop-drilling exceeds 3 levels. Likely never needed.

2.5 Real-Time System Design

Four Supabase Realtime subscriptions total:

#	Channel	Table	Event	Client Action
1	Global leaderboard	`profiles`	UPDATE on `total_score`	`cancelQueries` → `setQueryData` (update score, re-sort array) → `invalidateQueries` (full refetch for correctness)
2	Challenge leaderboard	`submissions`	INSERT, UPDATE on `score`	Same pattern as above
3	User notifications	`notifications`	INSERT where `user_id = current_user`	`setQueryData` (prepend to list, increment unread count)
4	Announcements	`announcements`	INSERT, UPDATE	`setQueryData` (add/update in ticker list)

Subscription lifecycle rules:

- Each subscription lives in a dedicated custom hook (e.g., `useRealtimeLeaderboard`)
- Subscriptions are created in `useEffect` with cleanup via `supabase.removeChannel()` on unmount
- Callbacks reference `queryClient` (stable singleton) to avoid stale closures
- Subscriptions are only active on pages that need them — not globally

Connection drop handling:

- Supabase Realtime has built-in reconnection with exponential backoff
- On `CHANNEL_ERROR` or `TIMED_OUT`: show a subtle toast "Live updates paused. Reconnecting..."
- On `SUBSCRIBED` (reconnection success): call `invalidateQueries` on all real-time queries to catch missed updates
- No manual polling fallback for the hackathon

Leaderboard update pattern (detailed):

1. Realtime event fires with the updated row
2. Call `queryClient.cancelQueries(['leaderboard'])` to stop any in-flight fetches
3. Call `queryClient.setQueryData(['leaderboard'], callback)` — the callback finds the user in the cached array, updates their score, and re-sorts. If the user isn't in the cached array, skip (the refetch will catch it).
4. Immediately call `queryClient.invalidateQueries(['leaderboard'])` — this triggers a background refetch from the server. When it resolves, it silently replaces the cache with the authoritative sorted list. This catches edge cases like users entering the top 100 from outside the cached set.
5. No `setTimeout`. No delay. The optimistic `setQueryData` prevents any loading flash. The `invalidateQueries` corrects silently in the background.

2.6 Caching Strategy

Page / Component	Method	Revalidation
Homepage (hero, stats, carousels)	ISR	`revalidate: 60` seconds
Team page	ISR	`revalidate: 3600` seconds
Finished events	ISR	`revalidate: 3600` seconds
Upcoming events	ISR	`revalidate: 60` seconds
Event detail page	ISR	`revalidate: 60` seconds
Projects listing	ISR	`revalidate: 300` seconds
Project detail	ISR	`revalidate: 300` seconds
Gallery albums	ISR	`revalidate: 3600` seconds
Blog listing	ISR	`revalidate: 300` seconds
Blog post detail	ISR	`revalidate: 300` seconds
Leaderboard	Client-side real-time	No SSR cache
Announcements ticker	Client-side real-time	No SSR cache
Notification bell	Client-side real-time	No SSR cache
Admin pages	Server-rendered, no cache	Always fresh
User dashboard	Server-rendered, no cache	Always fresh

Cache busting: After any admin mutation (create/edit/delete event, grade submission, etc.), the Server Action calls `revalidatePath('/events')` or the relevant path. This forces Next.js to regenerate the cached page on the next request.

2.7 Error Handling Architecture

Layer	Strategy
Global React error	`error.tsx` at root layout — styled fallback page with "Something went wrong" and a retry button
Page-level errors	`error.tsx` per route segment — scoped fallback UI
Server Action errors	Every Server Action returns `{ success: boolean, data?: T, error?: string }`. No raw throws. Client reads the response and shows appropriate feedback.
User-facing feedback	shadcn/ui Toast component. Success = green toast. Error = red toast. Info = blue toast. All toasts auto-dismiss after 5 seconds.
Loading states	`loading.tsx` per route — skeleton UI matching the page layout. Every mutation button shows a spinner during execution.
Network failure	If Supabase is unreachable, catch the error. Show "Something went wrong. Please try again." with a retry button. No raw error messages exposed.
Real-time connection	Toast on drop. Silent recovery on reconnect. No user action needed.

SECTION 3: TECH STACK — COMPLETE DEPENDENCY MAP

3.1 Core Dependencies

Package	Purpose	Why This One
`next` (14.x, App Router)	Full-stack React framework	SSR, ISR, API routes, middleware, image optimization. Native to Vercel. Best DX for our architecture.
`react`, `react-dom` (18.x)	UI library	Required by Next.js. Server Components support.
`typescript`	Type safety	Catches bugs at build time. Required for Supabase typed queries.
`tailwindcss`	Utility-first CSS	Rapid UI development. Design tokens configured in config file. Mobile-first responsive built-in.
`@tailwindcss/typography`	Prose styling	Renders rich text (blog posts, event descriptions) with consistent typography.
`framer-motion`	Animation library	Declarative React animations. Scroll-triggered entrances, page transitions, hover effects, stagger animations.
`@supabase/supabase-js`	Supabase client SDK	Database queries, auth, storage, real-time subscriptions from the browser.
`@supabase/ssr`	Supabase SSR helper	Cookie-based auth with PKCE flow. Creates server and browser Supabase clients. Session refresh middleware.

3.2 UI Component Dependencies

Package	Purpose
`shadcn/ui` (not an npm package — copied into project)	Pre-built accessible components: Dialog, Dropdown, Tabs, Tooltip, Popover, Select, Command, Toast, etc. Built on Radix. Styled with Tailwind. Fully customizable.
`@radix-ui/*` (installed as shadcn/ui dependencies)	Headless accessible primitives. Focus trapping, keyboard navigation, ARIA roles — all handled.
`lucide-react`	Icon library. Clean, consistent, tree-shakeable. Used by shadcn/ui by default.

3.3 Data & Form Dependencies

Package	Purpose
`@tanstack/react-query`	Server state management. Caching, refetching, optimistic updates, background sync. Integrates with our `initialData` convention.
`react-hook-form`	Form state management. Performant (no unnecessary re-renders). Integrates with Zod via resolver.
`zod`	Schema validation. Used for form validation (client) and Server Action validation (server). Shared schemas.
`@hookform/resolvers`	Connects Zod schemas to React Hook Form.

3.4 Rich Text & Content

Package	Purpose
`@tiptap/react`, `@tiptap/starter-kit`, `@tiptap/extension-link`	Rich text editor. Headless, ProseMirror-based, React 18 compatible. No hydration issues. Lazy-loaded — only on admin and blog creation pages.
`dompurify`	HTML sanitization. Strips malicious scripts from rich text before rendering. Configured with strict allowed tags list.
`@types/dompurify`	TypeScript types for DOMPurify.

3.5 Visualization & Media

Package	Purpose
`@react-three/fiber`	React renderer for Three.js. Used for the 3D Club Pulse visualization (Plan A).
`@react-three/drei`	Three.js helpers (OrbitControls, Html overlay, etc.). Used with R3F.
`three`	3D rendering engine. Peer dependency of R3F.
`d3-force-3d`	3D force-directed graph layout algorithm. Positions nodes for the Club Pulse visualization (Plan A only).
`recharts`	Chart library. Admin analytics dashboard — signups over time, submissions over time, active users. User performance graphs.

3.6 Utilities

Package	Purpose
`jspdf`	PDF generation in the browser. Used for on-demand certificate generation.
`html2canvas`	Renders HTML elements to canvas. Used with jsPDF to convert the certificate template to PDF.
`date-fns`	Date formatting and manipulation. Lightweight. Used for countdown timers, relative dates, heatmap calculations.
`clsx` + `tailwind-merge`	Conditional class name joining. Used throughout for dynamic Tailwind classes. (Installed as part of shadcn/ui setup.)

3.7 What Was Rejected and Why

Rejected	Why
React-Quill	Documented hydration issues with React 18 strict mode and SSR. Tiptap is safer for Next.js 14.
Firebase	NoSQL. Our data is heavily relational (users → submissions → challenges, events → registrations). PostgreSQL is the right model.
MongoDB Atlas	Same relational concern. Also requires separate auth and storage services, increasing complexity.
Netlify	Built for static sites. Next.js App Router features (middleware, Server Actions, ISR) require workarounds on Netlify. Vercel built Next.js.
react-pdf	Uses its own layout engine, not standard CSS. Designing a certificate requires learning a new API. jsPDF + html2canvas lets us design with HTML/CSS.
Zustand (pre-installed)	Not needed with our architecture. Auth Context + TanStack Query covers all state. Added only if prop-drilling exceeds 3 levels.
Email notifications (Resend/SendGrid)	Adds a third-party service, email templates, delivery monitoring. In-app notifications only for the hackathon.
Puppeteer (server-side PDF)	Doesn't run on Vercel serverless functions without significant workarounds. Client-side generation with jsPDF is instant.

SECTION 4: DATA MODEL

4.1 Table Definitions

All tables live in the `public` schema. All primary keys are `UUID` type with `gen_random_uuid()` default. All tables have `created_at TIMESTAMPTZ DEFAULT now()`. Tables with mutable content have `updated_at TIMESTAMPTZ DEFAULT now()`.

`profiles`

Extends Supabase Auth users. Created automatically via database trigger on signup.

Column	Type	Constraints	Notes
id	UUID	PK, references `auth.users(id)` ON DELETE CASCADE	Same ID as the auth user
full_name	TEXT	NOT NULL	
avatar_url	TEXT	NULLABLE	Supabase Storage URL
bio	TEXT	NULLABLE	Max 500 characters (enforced in app)
github_url	TEXT	NULLABLE	
linkedin_url	TEXT	NULLABLE	
tech_stack	TEXT[]	DEFAULT '{}'	Array of skill tags
role	TEXT	NOT NULL, DEFAULT 'member', CHECK (role IN ('member', 'admin'))	
total_score	INTEGER	NOT NULL, DEFAULT 0	Denormalized. Updated by PostgreSQL trigger on submissions.
current_streak	INTEGER	NOT NULL, DEFAULT 0	Login streak. Updated on each login.
longest_streak	INTEGER	NOT NULL, DEFAULT 0	
last_active_date	DATE	NULLABLE	Last login date in UTC. Used for streak calculation.

notify_event_reminders	BOOLEAN	DEFAULT true	
notify_challenge_deadlines	BOOLEAN	DEFAULT true	
notify_announcements	BOOLEAN	DEFAULT true	
last_dismissed_announcements_at	TIMESTAMPTZ	DEFAULT now()	For announcement dismissal
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

`team_members`

Curated showcase of the club's organizing body. NOT linked to auth signups. Admin manually manages.

Column	Type	Constraints	Notes
id	UUID	PK	
name	TEXT	NOT NULL	
photo_url	TEXT	NULLABLE	
designation	TEXT	NOT NULL	e.g., "President", "Technical Lead", "Member"
role_category	TEXT	NOT NULL, CHECK (role_category IN ('lead', 'core_team', 'member'))	For hierarchy display
batch_year	TEXT	NOT NULL	e.g., "2024", "2025"
domain	TEXT	NULLABLE	e.g., "Web Dev", "AI/ML", "Design"
skills	TEXT[]	DEFAULT '{}'	
github_url	TEXT	NULLABLE	
linkedin_url	TEXT	NULLABLE	
is_alumni	BOOLEAN	DEFAULT false	
alumni_current_role	TEXT	NULLABLE	Only relevant if is_alumni = true
alumni_current_company	TEXT	NULLABLE	Only relevant if is_alumni = true
display_order	INTEGER	NOT NULL, DEFAULT 0	For admin-controlled ordering
created_by	UUID	references `profiles(id)`	Which admin added this entry

created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

`events`

Column	Type	Constraints	Notes
id	UUID	PK	
title	TEXT	NOT NULL	
description	TEXT	NOT NULL	Rich text HTML from Tiptap
banner_image_url	TEXT	NULLABLE	
date_time	TIMESTAMPTZ	NOT NULL	Event start
end_date_time	TIMESTAMPTZ	NULLABLE	Event end
venue	TEXT	NULLABLE	
mode	TEXT	NOT NULL, CHECK (mode IN ('online', 'offline', 'hybrid'))	
category	TEXT	NOT NULL, CHECK (category IN ('workshop', 'hackathon', 'talk', 'competition', 'social'))	
status	TEXT	NOT NULL, DEFAULT 'draft', CHECK (status IN ('draft', 'upcoming', 'ongoing', 'finished'))	
max_participants	INTEGER	NULLABLE	NULL = unlimited
registration_count	INTEGER	NOT NULL, DEFAULT 0	Denormalized. Updated by trigger.
schedule	JSONB	NULLABLE	Array of {time, title,

			description}
speakers	JSONB	NULLABLE	Array of {name, role, image_url}
faqs	JSONB	NULLABLE	Array of {question, answer}
results	JSONB	NULLABLE	For finished events: {winners: [{name, position}], summary}
is_published	BOOLEAN	DEFAULT false	
is_deleted	BOOLEAN	DEFAULT false	Soft delete
created_by	UUID	references `profiles(id)`	
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

`event_registrations`

Column	Type	Constraints	Notes
id	UUID	PK	
event_id	UUID	NOT NULL, references `events(id)` ON DELETE CASCADE	
user_id	UUID	NOT NULL, references `profiles(id)` ON DELETE CASCADE	
status	TEXT	NOT NULL, DEFAULT 'registered', CHECK (status IN ('registered', 'attended', 'cancelled'))	Admin marks 'attended'. User can set 'cancelled'.
registered_at	TIMESTAMPTZ	DEFAULT now()	
UNIQUE(event_id, user_id)			One registration per user per event

`challenges`

Column	Type	Constraints	Notes
id	UUID	PK	
title	TEXT	NOT NULL	
description	TEXT	NOT NULL	Rich text HTML
problem_statement	TEXT	NOT NULL	Rich text HTML or markdown
difficulty	TEXT	NOT NULL, CHECK (difficulty IN ('easy', 'medium', 'hard'))	
category	TEXT	NOT NULL, CHECK (category IN ('ui_ux', 'full_stack', 'api_design', 'open_innovation'))	
status	TEXT	NOT NULL, DEFAULT 'draft', CHECK (status IN ('draft', 'active', 'closed'))	
start_date	TIMESTAMPTZ	NOT NULL	When the challenge becomes visible/submittable
deadline	TIMESTAMPTZ	NOT NULL	
max_score	INTEGER	NOT NULL, DEFAULT 100	
max_time_bonus	INTEGER	NOT NULL, DEFAULT 0	0 = no time bonus. Positive integer = max bonus points for early submission.

scoring_rules	JSONB	NULLABLE	Admin-defined criteria
tags	TEXT[]	DEFAULT '{}'	
is_deleted	BOOLEAN	DEFAULT false	
created_by	UUID	references `profiles(id)`	
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

`submissions`

Column	Type	Constraints	Notes
id	UUID	PK	
challenge_id	UUID	NOT NULL, references `challenges(id)` ON DELETE CASCADE	
user_id	UUID	NOT NULL, references `profiles(id)` ON DELETE CASCADE	
github_url	TEXT	NOT NULL	Validated format
live_url	TEXT	NULLABLE	Validated format if provided
score	INTEGER	NULLABLE	Set by admin during grading
time_bonus	INTEGER	NULLABLE	Calculated at grading time
feedback	TEXT	NULLABLE	Admin feedback
status	TEXT	NOT NULL, DEFAULT 'submitted', CHECK (status IN ('submitted', 'under_review', 'graded'))	
submitted_at	TIMESTAMPTZ	DEFAULT now()	
reviewed_by	UUID	NULLABLE, references `profiles(id)`	

reviewed_at	TIMESTAMPTZ	NULLABLE	

Note: A user CAN submit multiple times to the same challenge. Each submission is a new row. Admin grades the latest or best. No unique constraint on (challenge_id, user_id).

projects

Column	Type	Constraints	Notes
id	UUID	PK	
title	TEXT	NOT NULL	
description	TEXT	NOT NULL	Rich text HTML
thumbnail_url	TEXT	NULLABLE	
tech_stack	TEXT[]	DEFAULT '{}'	
team_members_names	TEXT[]	DEFAULT '{}'	Display names, not user references
github_url	TEXT	NULLABLE	
live_url	TEXT	NULLABLE	
documentation_url	TEXT	NULLABLE	
domain	TEXT	NULLABLE	
year	TEXT	NULLABLE	
team_size	INTEGER	NULLABLE	
is_featured	BOOLEAN	DEFAULT false	Shown on homepage
is_archived	BOOLEAN	DEFAULT false	
is_deleted	BOOLEAN	DEFAULT false	
likes_count	INTEGER	NOT NULL, DEFAULT 0	Denormalized. Updated by trigger.
screenshots	TEXT[]	DEFAULT '{}'	Array of Supabase Storage URLs
created_by	UUID	references	

		`profiles(id)`	
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

`project_likes`

Column	Type	Constraints	Notes
id	UUID	PK	
project_id	UUID	NOT NULL, references `projects(id)` ON DELETE CASCADE	
user_id	UUID	NOT NULL, references `profiles(id)` ON DELETE CASCADE	
created_at	TIMESTAMPTZ	DEFAULT now()	
UNIQUE(project_id, user_id)			One like per user per project

`gallery_albums`

Column	Type	Constraints	Notes
id	UUID	PK	
title	TEXT	NOT NULL	
event_id	UUID	NULLABLE, references `events(id)`	Not all albums are tied to an event
event_date	DATE	NULLABLE	For display and sorting
cover_image_url	TEXT	NULLABLE	
description	TEXT	NULLABLE	
created_by	UUID	references `profiles(id)`	
created_at	TIMESTAMPTZ	DEFAULT now()	

`gallery_images`

Column	Type	Constraints	Notes
id	UUID	PK	
album_id	UUID	NOT NULL, references `gallery_albums(id)` ON DELETE CASCADE	
image_url	TEXT	NOT NULL	Supabase Storage URL
caption	TEXT	NULLABLE	
display_order	INTEGER	NOT NULL, DEFAULT 0	
created_at	TIMESTAMPTZ	DEFAULT now()	

`notifications`

Column	Type	Constraints	Notes
id	UUID	PK	
user_id	UUID	NOT NULL, references `profiles(id)` ON DELETE CASCADE	Always per-user. No broadcast rows.
title	TEXT	NOT NULL	
message	TEXT	NOT NULL	
type	TEXT	NOT NULL, CHECK (type IN ('event_reminder', 'challenge_deadline', 'badge_earned', 'submission_graded'))	
is_read	BOOLEAN	DEFAULT false	
link	TEXT	NULLABLE	URL to navigate to when clicked
created_at	TIMESTAMPTZ	DEFAULT now()	

`announcements`

Column	Type	Constraints	Notes
id	UUID	PK	
title	TEXT	NOT NULL	
content	TEXT	NOT NULL	
is_active	BOOLEAN	DEFAULT true	
created_by	UUID	references `profiles(id)`	
created_at	TIMESTAMPTZ	DEFAULT now()	

`badges`

Column	Type	Constraints	Notes
id	UUID	PK	
name	TEXT	NOT NULL, UNIQUE	e.g., "First Submission", "Legend"
description	TEXT	NOT NULL	
icon_url	TEXT	NULLABLE	Badge image
criteria_type	TEXT	NOT NULL	e.g., "first_submission", "total_score_1000", "challenges_completed_5", "events_attended_3"
criteria_value	INTEGER	NULLABLE	Threshold value for count-based badges
created_at	TIMESTAMPTZ	DEFAULT now()	

`user_badges`

Column	Type	Constraints	Notes
id	UUID	PK	
user_id	UUID	NOT NULL, references `profiles(id)` ON DELETE CASCADE	
badge_id	UUID	NOT NULL, references `badges(id)` ON DELETE CASCADE	
earned_at	TIMESTAMPTZ	DEFAULT now()	
UNIQUE(user_id, badge_id)			One badge per user

`blog_posts`

Column	Type	Constraints	Notes
id	UUID	PK	
title	TEXT	NOT NULL	
content	TEXT	NOT NULL	Rich text HTML
author_id	UUID	NOT NULL, references `profiles(id)` ON DELETE CASCADE	
cover_image_url	TEXT	NULLABLE	
tags	TEXT[]	DEFAULT '{}'	
status	TEXT	NOT NULL, DEFAULT 'draft', CHECK (status IN ('draft', 'published'))	
is_deleted	BOOLEAN	DEFAULT false	
published_at	TIMESTAMPTZ	NULLABLE	Set when status changes to 'published'
created_at	TIMESTAMPTZ	DEFAULT now()	
updated_at	TIMESTAMPTZ	DEFAULT now()	

`certificates`

Column	Type	Constraints	Notes
id	UUID	PK	
event_id	UUID	NOT NULL, references `events(id)`	
user_id	UUID	NOT NULL, references `profiles(id)`	
certificate_type	TEXT	NOT NULL, CHECK (certificate_type IN ('participation', 'winner'))	
downloaded_at	TIMESTAMPTZ	NULLABLE	Set when user downloads
created_at	TIMESTAMPTZ	DEFAULT now()	
UNIQUE(event_id, user_id, certificate_type)			One cert per type per user per event

`admin_audit_log`

Column	Type	Constraints	Notes
id	UUID	PK	
admin_user_id	UUID	NOT NULL, references `profiles(id)`	
action	TEXT	NOT NULL	e.g., "created_event", "graded_submission", "promoted_user"
entity_type	TEXT	NOT NULL	e.g., "event", "submission", "profile"
entity_id	UUID	NULLABLE	
details	JSONB	NULLABLE	Additional context
created_at	TIMESTAMPTZ	DEFAULT now()	

4.2 PostgreSQL Triggers

Trigger 1: Auto-create profile on signup

- Fires: `AFTER INSERT ON auth.users`
- Action: Inserts a new row into `profiles` with the user's ID and email-derived name
- Purpose: Every auth user always has a corresponding profile row

Trigger 2: Update event registration count

- Fires: `AFTER INSERT OR DELETE ON event_registrations`
- Action: Recalculates `COUNT(*)` from `event_registrations` where `event_id = NEW.event_id` (or `OLD.event_id` on delete) and `status != 'cancelled'`, then updates `events.registration_count`
- Purpose: Keeps the denormalized count accurate

Trigger 3: Update project likes count

- Fires: `AFTER INSERT OR DELETE ON project_likes`
- Action: Recalculates `COUNT(*)` from `project_likes` where `project_id = NEW.project_id`, then updates `projects.likes_count`
- Purpose: Keeps the denormalized count accurate

Trigger 4: Update user total score

- Fires: `AFTER INSERT OR UPDATE OF score, time_bonus ON submissions`
- Action: Recalculates `SUM(COALESCE(score, 0) + COALESCE(time_bonus, 0))` from `submissions` where `user_id = NEW.user_id` and `status = 'graded'`, then updates `profiles.total_score`
- Purpose: Leaderboard accuracy. This is the most critical trigger.

4.3 Database Views

All views use `WITH (security_invoker = true)` to respect RLS policies of the calling user.

View Name	Definition	Used By
`active_events`	`SELECT * FROM events WHERE is_deleted = false`	All public and member queries
`active_challenges`	`SELECT * FROM challenges WHERE is_deleted = false`	All public and member queries
`active_projects`	`SELECT * FROM projects WHERE is_deleted = false`	All public and member queries
`active_blog_posts`	`SELECT * FROM blog_posts WHERE is_deleted = false`	All public and member queries

After creating views, grant access: `GRANT SELECT ON [view_name] TO anon, authenticated;`

Admin queries use the raw tables directly (with a filter toggle to show/hide deleted items).

4.4 Row Level Security Policies

RLS is enabled on EVERY table. The principle: the database enforces access rules regardless of what the frontend sends.

profiles:

- SELECT: Everyone can read any profile (public profiles)

- UPDATE: Only the profile owner can update their own row. Admins can update any row.
- INSERT: Only the trigger (service role) inserts profiles
- DELETE: Not allowed through the API

team_members:

- SELECT: Everyone can read (public page)
- INSERT, UPDATE, DELETE: Only admins

events / active_events:

- SELECT: Published events visible to everyone. Drafts visible only to admins.
- INSERT, UPDATE, DELETE: Only admins

event_registrations:

- SELECT: Users can read their own registrations. Admins can read all.
- INSERT: Authenticated users can register themselves (user

_id must match auth.uid())

- UPDATE: Users can cancel their own registration. Admins can update any (for marking attendance).
- DELETE: Not allowed through API. Use status = 'cancelled' instead.

challenges / active_challenges:

- SELECT: Active and closed challenges visible to everyone. Drafts visible only to admins.
- INSERT, UPDATE, DELETE: Only admins

submissions:

- SELECT: Users can read their own submissions. Admins can read all.
- INSERT: Authenticated users can insert where user_id matches auth.uid()
- UPDATE: Only admins (for grading — setting score, feedback, status, reviewed_by)
- DELETE: Not allowed through API

projects / active_projects:

- SELECT: Non-archived projects visible to everyone. Archived visible only to admins.
- INSERT, UPDATE, DELETE: Only admins

project_likes:

- SELECT: Everyone can read (for like counts)
- INSERT: Authenticated users where user_id matches auth.uid()
- DELETE: Users can remove their own like. user_id must match auth.uid()

gallery_albums:

- SELECT: Everyone can read
- INSERT, UPDATE, DELETE: Only admins

gallery_images:

- SELECT: Everyone can read
- INSERT, UPDATE, DELETE: Only admins

notifications:

- SELECT: Users can only read their own (user_id = auth.uid())
- INSERT: Only service role (server-side actions create notifications)
- UPDATE: Users can update their own (marking as read)
- DELETE: Users can delete their own

announcements:

- SELECT: Active announcements visible to everyone
- INSERT, UPDATE, DELETE: Only admins

badges:

- SELECT: Everyone can read
- INSERT, UPDATE, DELETE: Only admins

user_badges:

- SELECT: Everyone can read (badges are public on profiles)
- INSERT: Only service role (badge award logic runs server-side)
- DELETE: Not allowed through API

blog_posts / active_blog_posts:

- SELECT: Published posts visible to everyone. Drafts visible only to the author and admins.
- INSERT: Authenticated users where author_id matches auth.uid()
- UPDATE: Authors can update their own posts. Admins can update any.
- DELETE: Only admins (soft delete)

certificates:

- SELECT: Users can read their own. Admins can read all.
- INSERT: Only service role (created when user downloads)
- UPDATE: Only service role (updating downloaded_at)
- DELETE: Not allowed

admin_audit_log:

- SELECT: Only admins
- INSERT: Only service role
- UPDATE, DELETE: Not allowed

How "only admins" is enforced in RLS:

The RLS policy checks `(SELECT role FROM profiles WHERE id = auth.uid()) = 'admin'`. This subquery runs within the database — no way to bypass from the client.

SECTION 5: AUTHENTICATION & SECURITY ARCHITECTURE

5.1 Auth Flow — Step by Step

Signup (Email/Password):

1. User fills signup form: email, password, full name
2. Client calls Supabase Auth `signUp({ email, password, options: { data: { full_name } } })`
3. Supabase creates the user in `auth.users`
4. The database trigger fires, creating a `profiles` row with the user's ID and full_name
5. Default role is 'member'
6. Session is created, cookies are set via `@supabase/ssr`
7. User is redirected to the `redirect` query parameter URL, or `/dashboard` if none exists

Signup (OAuth — Phase 7B, if time allows):

1. Before initiating OAuth, store the `redirect` URL (from query params) in `localStorage`
2. Call `supabase.auth.signInWithOAuth({ provider: 'google' })` or `'github'`
3. User is redirected to Google/GitHub, authorizes, returns to callback URL
4. Supabase handles the token exchange
5. Database trigger creates the profile row if it's a new user
6. Middleware refreshes the session
7. Client reads `redirect` from localStorage, validates it starts with `/`, navigates there, clears localStorage

Login:

1. User enters email and password
2. Client calls `supabase.auth.signInWithPassword({ email, password })`
3. Session cookies set via `@supabase/ssr`
4. Redirect to `redirect` query param or `/dashboard`

Session Refresh:

Next.js middleware runs on every request. It creates a Supabase server client, calls

`supabase.auth.getUser()`, which reads and refreshes the session cookies if needed. This ensures Server Components always have access to the current session.

Logout:

1. Client calls `supabase.auth.signOut()`
2. Cookies cleared
3. Redirect to `/`

5.2 Route Protection

Three levels of protection:

Level	Implementation	Protected Routes
Public	No protection. Anyone can access.	`/`, `/team`, `/events`, `/events/[id]`, `/challenges`, `/challenges/[id]`, `/projects`, `/projects/[id]`, `/gallery`, `/gallery/[albumId]`, `/leaderboard`, `/blog`, `/blog/[id]`, `/profile/[id]`, `/login`, `/signup`
Member	Middleware checks for valid session. Redirects to `/login` if no session.	`/dashboard`, `/profile/settings`, `/challenges/[id]/submit`, `/blog/new`
Admin	Middleware checks for valid session AND queries `profiles.role = 'admin'`. Redirects to `/` if not admin.	`/admin`, `/admin/*`

Middleware implementation:

The middleware runs on every request matching the protected route patterns. It does NOT run on static assets, images, or public routes (configured via `matcher` in `middleware.ts`).

5.3 Security Measures

Threat	Mitigation
XSS (Cross-Site Scripting)	All rich text content sanitized with DOMPurify before rendering. HTTP-only cookies prevent token theft via script injection.
CSRF (Cross-Site Request Forgery)	Supabase PKCE flow includes CSRF protection. Server Actions are POST-only with origin checking.
SQL Injection	Supabase client uses parameterized queries. No raw SQL from user input.
Unauthorized data access	RLS on every table. Even if frontend code is bypassed, the database rejects unauthorized operations.
Privilege escalation	Role changes only through admin Server Action that verifies the caller is admin before modifying `profiles.role`.
File upload abuse	Client-side validation: only .jpg, .jpeg, .png, .webp. Max 5MB (2MB for avatars). Server-side: Supabase Storage bucket policies enforce file type and size limits.
Brute force login	Supabase Auth built-in rate limiting on auth endpoints. Sufficient for hackathon.
Token theft	Tokens in HTTP-only cookies. Not accessible via JavaScript. Not stored in localStorage.
Malicious redirect	OAuth redirect URL validated: must start with `/` (relative path on our domain). Absolute URLs ignored.
Admin impersonation	First admin seeded directly in database. Promotion requires existing admin Server Action. No self-promotion possible.

5.4 DOMPurify Configuration

Exact configuration used whenever rendering rich text HTML:

Allowed tags: p, h1, h2, h3, strong, em, u, a, ul, ol, li, blockquote, code, pre, hr, br

Allowed attributes: href, target, rel (only on <a> tags)

Explicitly blocked: <img>, <iframe>, <script>, <style>, <video>, <audio>, <form>, <input>, all on* event attributes

If images are needed in blog posts in the future, they go through the platform's file upload flow and are inserted as safe <img> tags with Supabase Storage URLs only.

SECTION 6: DESIGN SYSTEM

6.1 Color Palette with Usage Rules

Primary Brand Colors

Name	Hex	Role	Usage Rules
Deep Navy Blue	#0B3A63	Primary dark background	Hero sections, navigation, footer, dark cards. ALL text on this must be #FFFFFF or #D9E2EC.
Steel Blue	#1E5A8A	Secondary dark background	Card backgrounds on dark sections, secondary panels. ALL text on this must be #FFFFFF or #D9E2EC.
Soft Blue Grey	#D9E2EC	Light accent on dark, secondary text on dark	Borders on dark backgrounds, muted text on dark backgrounds. NEVER used as text on light backgrounds (insufficient contrast).

Energy Colors

Name	Hex	Role	Usage Rules
Bright Red	#FC0204	Primary CTA	Primary buttons, critical actions. Text ON this button must be #FFFFFF.
Crimson Red	#E63946	CTA hover / secondary accent	Button hover states, error states. Text must be #FFFFFF.
Deep Maroon	#640204	Active states on dark	Button active/pressed state, borders on interactive elements on dark backgrounds.
Soft Salmon Pink	#FCA2A4	Soft accent	Tags, soft badges, light highlights on white backgrounds. Text ON this must be #222222.

Highlight Colors

Name	Hex	Role	Usage Rules
Bright Yellow	#FCF604	Attention, featured badges	Star ratings, "Featured" badges, attention indicators. ONLY on dark backgrounds (#0B3A63, #1E5A8A). NEVER on or near white. Text ON this must be #222222.
Golden Orange	#F5A623	Secondary CTA on dark, notifications	CTA buttons on dark backgrounds (alternative to red), notification count badge, achievement highlights. Text ON this must be #222222.
Olive Gold	#8C8604	Subtle highlight	Rank indicators, muted emphasis. Use sparingly. Only on dark backgrounds.

Neutrals

Name	Hex	Role	Usage Rules
Pure White	#FFFFFF	Light surface	Card surfaces, modals, input fields, page backgrounds for light sections. Text must be #222222 or #6C757D.
Light Grey	#F4F6F8	Page background	Full page background for light pages, alternating section backgrounds. Text must be #222222.

Text Colors

Name	Hex	Usage Rules
Charcoal Black	#222222	Primary text ON light backgrounds ONLY (#FFFFFF, #F4F6F8, #FCA2A4, #FCF604). NEVER on dark backgrounds.
Muted Grey	#6C757D	Secondary text, captions, timestamps ON light backgrounds ONLY. NEVER on dark backgrounds.
Cool Grey	#A0AEC0	Placeholder text, disabled states ON light backgrounds ONLY.

Contrast Rules (Non-Negotiable)

These rules are absolute. No exceptions for any page, component, or state:

1. Text on #0B3A63 (Navy): ONLY #FFFFFF or #D9E2EC
2. Text on #1E5A8A (Steel Blue): ONLY #FFFFFF or #D9E2EC
3. Text on #FFFFFF (White): ONLY #222222, #6C757D, or #A0AEC0
4. Text on #F4F6F8 (Light Grey): ONLY #222222 or #6C757D
5. Text on #FC0204 (Red buttons): ONLY #FFFFFF
6. Text on #FCA2A4 (Salmon): ONLY #222222
7. Text on #FCF604 (Yellow): ONLY #222222
8. #FCF604 NEVER appears on or adjacent to #FFFFFF or #F4F6F8
9. #6C757D and #A0AEC0 NEVER used as text on dark backgrounds
10. #222222 NEVER used on dark backgrounds

6.2 Typography System

Role	Font Family	Weights	Usage
Display / Hero	Space Grotesk	700	Large cinematic headings on hero sections only. Homepage title, major section headers.
Headings	Space Grotesk	600, 700	h1–h3 across all pages. Clean, modern, geometric.
Body	Inter	400, 500	All paragraph text, descriptions, form labels, navigation links.
Data / Stats / Code	JetBrains Mono	400, 500	Leaderboard numbers, scores, countdown timers, code snippets, badge labels, stat counters. Gives the tech/hacker aesthetic.

Type Scale (responsive):

Element	Mobile	Desktop
h1 (page title)	2rem (32px)	3.5rem (56px)
h2 (section title)	1.5rem (24px)	2.5rem (40px)
h3 (card title)	1.25rem (20px)	1.75rem (28px)
Body	1rem (16px)	1.125rem (18px)
Caption / Small	0.75rem (12px)	0.875rem (14px)
Stat number (JetBrains Mono)	2rem (32px)	3rem (48px)

Line heights:

- Headings: 1.2
- Body: 1.6
- Captions: 1.4

6.3 Spacing System

Base unit: 4px (Tailwind default)

Token	Tailwind Class	Value	Usage
xs	`space-1` / `p-1`	4px	Tight gaps between inline elements
sm	`space-2` / `p-2`	8px	Icon-to-text gap, badge padding
md	`space-3` / `p-3`	12px	Card internal padding (mobile)
base	`space-4` / `p-4`	16px	Card internal padding (desktop), form field spacing
lg	`space-6` / `p-6`	24px	Between cards in a grid
xl	`space-8` / `p-8`	32px	Section header to content
2xl	`space-12` / `p-12`	48px	Between major sections on a page
3xl	`space-16` / `p-16`	64px	Hero section vertical padding
4xl	`space-24` / `p-24`	96px	Between homepage mega-sections

Max content width: 1280px (`max-w-7xl`). Centered with `mx-auto`.

Grid: 12-column CSS Grid on desktop. Single column on mobile.

Breakpoints (Tailwind defaults):

Name	Width	Usage
sm	640px	Small tablets, large phones landscape
md	768px	Tablets portrait
lg	1024px	Small laptops, tablets landscape
xl	1280px	Standard desktops

6.4 Card Design Tokens

Property	Value
Border radius	12px (`rounded-xl`)
Shadow (resting)	`shadow-md`
Shadow (hover)	`shadow-xl` with 300ms transition
Background (light section)	`#FFFFFF`
Background (dark section)	`#1E5A8A`
Padding	16px mobile (`p-4`), 24px desktop (`p-6`)
Border	None by default. 1px `#D9E2EC` on light backgrounds where separation is needed.
Image aspect ratio (in cards)	16:9 for banners, 1:1 for avatars

6.5 Animation System

Durations:

Token	Value	Usage
fast	150ms	Button hover, toggle, small state changes
normal	300ms	Card hover effects, modal open/close, dropdown, toast appear
slow	500ms	Page transitions, section fade-ins, carousel slides
hero	800ms	Hero section entrance, 3D/2D scene initial animation

Easing Curves:

Token	Value	Usage
ease-out	`cubic-bezier(0.16, 1, 0.3, 1)`	Entrances — elements coming into view
ease-in	`cubic-bezier(0.55, 0, 1, 0.45)`	Exits — elements leaving view
ease-in-out	`cubic-bezier(0.65, 0, 0.35, 1)`	State transitions (tabs, toggles)
spring	Framer Motion `type: "spring", stiffness: 300, damping: 30`	Interactive elements (cards lift, buttons press)

Scroll-Triggered Animations:

- Trigger: element 20% in viewport (`viewport: { once: true, amount: 0.2 }`)
- Pattern: fade up — `opacity: 0 → 1`, `y: 30px → 0`, duration `slow`, easing `ease-out`

- All scroll animations trigger ONCE. No repeat on scroll back.

Stagger for Lists:

- Delay between items: 80ms
- Max stagger: 8 items (640ms total)
- Items beyond 8 appear with the 8th item's timing

Page Transitions:

- Type: Crossfade
- Duration: `slow` (500ms)
- `opacity: 0 → 1` with `ease-out`
- No slide transitions — slides feel heavy on data-dense pages

What Does NOT Animate:

- Body text paragraphs
- Data table content
- Form labels and inputs
- Navigation text links
- Any content that the user needs to read immediately

6.6 Design Principles

1. **Dark-first:** The primary experience is dark (navy backgrounds). Light sections are used for contrast and variety, not as a separate "mode." There is no light/dark mode toggle. The site is designed as one cohesive dark-themed experience with light cards and sections for visual rhythm.
2. **Cinematic composition:** Large hero sections with dramatic gradient overlays. Bold typography. Generous whitespace. The feeling of a movie poster, not a dashboard.
3. **Sci-fi texture:** Subtle glow effects on borders (box-shadow with brand colors at low opacity). Monospace font for data elements. Geometric background patterns (subtle CSS-generated, not images). The Matrix-style element confined to the loading/splash animation or hero background — NOT sitewide.
4. **Mobile-first:** Every layout designed for mobile first, expanded for desktop. Touch targets minimum 44x44px. No hover-only interactions — everything accessible via tap.
5. **Breathing room:** Generous padding (see spacing tokens). Cinematic visuals need room to breathe. Cramped layouts kill the aesthetic.
6. **Micro-interactions:** Every button has a hover state. Every card lifts on hover. Every interactive element gives tactile feedback. But animations serve the interaction, not distract from it.
7. **Consistent card language:** All listing pages (events, challenges, projects, team) use the same card design tokens. The content varies but the visual container is recognizable.

6.7 Accessibility Requirements

Requirement	Implementation
Color contrast	WCAG AA minimum (4.5:1 for body text, 3:1 for large text). Enforced by the contrast rules in 6.1.
ARIA labels	All icon-only buttons have `aria-label`. Notification bell, hamburger menu, close buttons, carousel controls.
Keyboard navigation	All interactive elements reachable via Tab. Enter/Space activates buttons. Escape closes modals/dropdowns.
Focus trapping	Modals and lightbox trap focus using Radix Dialog. Tab cycles within, not behind.
Skip link	Hidden "Skip to main content" link at top of every page, visible on keyboard focus.
Three.js / 2D viz	Wrapped in `role="img"` with descriptive `aria-label`. Not used for critical information.
Screen reader	All images have `alt` text. Decorative images use `alt=""`. Status messages use `aria-live="polite"`.
Reduced motion	Respect `prefers-reduced-motion` media query. Disable Framer Motion animations and Three.js animation loop when active.

SECTION 7: COMPLETE ROUTE MAP

7.1 Public Routes

Route	Page	Data Source	Cache
`/`	Homepage	Server Component. Fetches: stats counts, upcoming events (3), featured challenges (3), featured projects (3), top 5 leaderboard, active announcements.	ISR 60s
`/team`	Team Section	Server Component. Fetches: all team_members ordered by role_category then display_order.	ISR 3600s
`/events`	Events Listing	Server Component. Fetches: active_events where is_published=true, sorted by date_time. Tabbed: upcoming/finished.	ISR 60s
`/events/[id]`	Event Detail	Server Component. Fetches: single event by ID with registration count.	ISR 60s
`/challenges`	Challenge Listing	Server Component. Fetches: active_challenges where status IN ('active','closed'), sorted by deadline.	ISR 60s
`/challenges/[id]`	Challenge Detail	Server Component. Fetches: single	ISR 60s

		challenge by ID. If authenticated, also fetches user's submissions for this challenge.	
`/projects`	Projects Listing	Server Component. Fetches: active_projects where is_archived=false, sorted by is_featured DESC then created_at DESC.	ISR 300s
`/projects/[id]`	Project Detail	Server Component. Fetches: single project by ID. If authenticated, also fetches user's like status.	ISR 300s
`/gallery`	Gallery Albums	Server Component. Fetches: all gallery_albums with image count per album, sorted by event_date DESC.	ISR 3600s
`/gallery/[albumId]`	Album Images	Server Component. Fetches: album details + all gallery_images for this album, ordered by display_order.	ISR 3600s
`/leaderboard`	Global Leaderboard	Client Component. Fetches via TanStack Query on mount. Supabase Realtime	No SSR cache

		subscription for live updates.	
`/blog`	Blog Listing	Server Component. Fetches: active_blog_posts where status='published', sorted by published_at DESC.	ISR 300s
`/blog/[id]`	Blog Post Detail	Server Component. Fetches: single blog post by ID. Content sanitized with DOMPurify.	ISR 300s
`/profile/[id]`	Public User Profile	Server Component. Fetches: profile, user_badges, public submission stats.	ISR 300s
`/login`	Login Page	No data fetch. Form only. Reads `redirect` query param.	Static
`/signup`	Signup Page	No data fetch. Form only. Reads `redirect` query param.	Static

7.2 Member Routes (Auth Required)

Route	Page	Data Source	Cache
`/dashboard`	Personal Dashboard	Server Component. Fetches: own profile, own submissions with scores, own event registrations, own badges, derived activity heatmap query.	No cache
`/profile/settings`	Account Settings	Server Component. Fetches: own profile. Form pre-filled.	No cache
`/challenges/[id]/submit`	Submit Solution	Server Component. Fetches: challenge details (for context). Form for submission.	No cache
`/blog/new`	Create Blog Post	Client Component. Tiptap editor loaded dynamically.	No cache
`/blog/edit/[id]`	Edit Blog Post	Client Component. Fetches own post. Tiptap editor loaded dynamically.	No cache

7.3 Admin Routes (Admin Auth Required)

Route	Page
`/admin`	Admin Dashboard — analytics charts, quick stats
`/admin/events`	Event management — list, create, edit, soft-delete, publish/unpublish
`/admin/events/new`	Create event form
`/admin/events/[id]/edit`	Edit event form
`/admin/challenges`	Challenge management — list, create, edit, soft-delete
`/admin/challenges/new`	Create challenge form
`/admin/challenges/[id]/edit`	Edit challenge form
`/admin/submissions`	Submission review — list all, filter by challenge, grade, feedback
`/admin/team`	Team member management — list, add, edit, remove, reorder
`/admin/team/new`	Add team member form
`/admin/team/[id]/edit`	Edit team member form
`/admin/projects`	Project management — list, add, edit, feature, archive, soft-delete
`/admin/projects/new`	Add project form
`/admin/projects/[id]/edit`	Edit project form
`/admin/gallery`	Gallery management — list albums, create, edit, delete
`/admin/gallery/[albumId]`	Album image management — upload, reorder, delete images

`/admin/users`	User management — list all, search, promote to admin, ban
`/admin/announcements`	Announcement management — create, toggle active, delete
`/admin/audit-log`	Audit log viewer — filterable, paginated

SECTION 8: FEATURE SPECIFICATIONS

This section provides the complete specification for every feature. Each specification includes what the user sees, what happens when they interact, and what data flows occur.

8.1 Homepage

Components on the page (top to bottom):

1. **Hero Section**
   - Full-viewport-height section with Deep Navy (#0B3A63) background
   - Club name "BOOT FORCE" in display typography (Space Grotesk 700)
   - Tagline below the name
   - Two CTA buttons: "Join Boot Force" (red, links to `/signup`) and "Explore" (outlined, scrolls to next section)
   - The Club Pulse visualization (3D or 2D depending on Plan A/B success) positioned as background or side element
   - Entrance animation: text fades up, buttons fade up with stagger, visualization fades in
2. **Stats Bar**
   - Horizontal strip with 4 counters: Members, Events Hosted, Challenges, Projects Built
   - Numbers animate from 0 to actual value on scroll into view (counting animation, JetBrains Mono font)
   - Data: fetched server-side, passed as props
   - Values come from: `COUNT(*)` on profiles (members), events where status='finished' (events hosted), challenges (total), projects (total)
3. **Upcoming Events Strip**
   - Section title: "Upcoming Events"
   - Horizontal scrollable row of event cards (3 max)
   - Each card: title, category tag, date, countdown timer
   - "View All Events" link at the end
   - If no upcoming events: "No upcoming events. Stay tuned!" message
4. **Featured Challenges Carousel**

- Section title: "Active Challenges"
- Carousel of challenge cards (3 max, auto-rotating, pausable)
- Each card: title, difficulty badge (color-coded: green/yellow/red), category, deadline
- "View All Challenges" link

5. **Featured Projects Carousel**
   - Section title: "Our Projects"
   - Carousel of project cards (3 max, where is_featured=true)
   - Each card: thumbnail, title, tech stack tags, likes count
   - "View All Projects" link
6. **Top 5 Leaderboard Preview**
   - Section title: "Top Performers"
   - Compact table: rank, avatar, name, score, rank badge
   - "View Full Leaderboard" link
   - This section uses server-fetched data (NOT real-time on the homepage — real-time is only on the full leaderboard page)
7. **Announcements Ticker**
   - Scrolling horizontal marquee of active announcements
   - Client Component with Supabase Realtime subscription
   - If no announcements: section hidden
8. **Footer**
   - Club logo and name
   - Quick links: Team, Events, Challenges, Projects, Gallery, Blog
   - Social media links (GitHub, LinkedIn, Instagram, Twitter)
   - "Join Boot Force" CTA button
   - "© 2025 Boot Force — CMR University" copyright text

8.2 Navigation Bar

Present on every page. Responsive.

Desktop (≥1024px):

- Left: Boot Force logo + name (links to /)
- Center: nav links — Home, Team, Events, Challenges, Projects, Gallery, Leaderboard, Blog
- Right (unauthenticated): "Sign In" button
- Right (authenticated): Notification bell (with unread count badge) + User avatar dropdown (Profile, Dashboard, Settings, Logout). If admin: "Admin Panel" link in dropdown.

Mobile (<1024px):

- Left: Boot Force logo
- Right: Notification bell (if authenticated) + Hamburger menu icon
- Hamburger opens a full-screen overlay with all nav links, auth links, and user info
- Menu closes on link click or close button

Notification Bell:

- Client Component. Real-time subscription on `notifications` table + `announcements` query.
- Shows unread count as a red badge (#FC0204)
- Click opens a dropdown panel showing recent notifications
- Each notification: icon (by type), title, message, time ago, link
- "Mark all as read" button at top
- Click a notification: marks as read, navigates to the link
- "Dismiss all announcements" updates `profiles.last_dismissed_announcements_at`

8.3 Team Section

URL: `/team`

Layout:

- Page title: "Our Team"
- Filter bar: dropdowns for Role Category (All, Leads, Core Team, Members), Batch Year, Domain
- Alumni toggle switch
- Below filters: sections for each role category, in order: Leads → Core Team → Members
- Each section has a header and a grid of cards

Team Member Card:

- Photo (circular, 120px)
- Name (h3)
- Designation
- Batch year
- Domain badge
- Skill tags (horizontal scrollable)
- Social links (GitHub, LinkedIn icons)
- Hover: card lifts (`shadow-xl`), subtle scale (1.02)
- Click: opens a Modal with full details — larger photo, full bio/skills, social links

Alumni View:

- When alumni toggle is ON, the grid shows alumni members
- Alumni cards have additional fields: Current Role, Current Company
- Visually distinguished with a subtle "Alumni" badge on the card

Admin Actions (from `/admin/team`):

- Add new team member: form with all fields + photo upload
- Edit existing: pre-filled form
- Remove: confirmation dialog → hard delete
- Reorder: drag-and-drop or manual display_order number input

8.4 Events System

URL: `/events`

Layout:

- Page title: "Events"
- Two tabs: "Upcoming" and "Past"
- Search bar: searches event titles (server-side ILIKE)
- Category filter: All, Workshop, Hackathon, Talk, Competition, Social

Upcoming Events Tab:

- Grid of event cards, sorted by date_time ascending (soonest first)
- Each card: banner image (16:9), title, category badge, date and time (formatted), venue/mode, registration count, countdown timer (live ticking)
- Countdown timer: shows Days:Hours:Minutes:Seconds, updates every second via `setInterval`
- Click card → navigate to `/events/[id]`

Past Events Tab:

- Same card layout but no countdown timer
- Shows results snippet if available (winner names)
- "View Gallery" link if a gallery album is linked

Event Detail Page (`/events/[id]`):

- Banner image (full width)
- Title, category badge, date/time, venue/mode
- Registration count and countdown (if upcoming)
- "Register" button:
  - If visitor: shows "Sign in to register" → navigates to `/login?redirect=/events/[id]`
  - If member (not registered): shows "Register" → calls Server Action → success toast → button changes to "Registered ✓"
  - If member (already registered): shows "Registered ✓" with option to cancel
  - If event is finished or full: button disabled with appropriate message
- Description section: rich text rendered with DOMPurify + Tailwind prose
- Schedule: rendered from JSON — timeline format with time, title, description per slot
- Speakers: cards with name, role, image
- FAQs: accordion component (shadcn/ui)
- If finished: Results section showing winners and summary
- If gallery album linked: "View Event Gallery" button

Admin Event Management:

- Create: form with title, rich text editor (Tiptap) for description, image upload for banner, date pickers, venue, mode select, category select, max participants, schedule builder (add/remove slots), speakers builder (add/remove), FAQs builder (add/remove)
- Edit: same form, pre-filled
- Publish/Unpublish: toggle switch
- Soft delete: confirmation dialog → sets is_deleted=true
- Mark attendance: on a finished event, see list of registrations with checkboxes. "Mark as Attended" per person or "Mark All as Attended" bulk button

8.5 Challenge & Competition System

URL: `/challenges`

Layout:

- Page title: "Challenges"
- Filter bar: Difficulty (All, Easy, Medium, Hard), Category (All, UI/UX, Full-Stack, API Design, Open Innovation), Status (All, Active, Closed)
- Search bar: searches challenge titles

Challenge Cards:

- Title
- Difficulty badge: Easy = green, Medium = yellow (#F5A623), Hard = red (#FC0204)
- Category badge
- Deadline (formatted, with "time remaining" relative text)
- Submission count
- Status badge: Active = green dot, Closed = grey dot
- Click → navigate to `/challenges/[id]`

Challenge Detail Page (`/challenges/[id]`):

- Title, difficulty badge, category badge, deadline with countdown
- Problem statement: rich text rendered with DOMPurify + prose styling
- Scoring rules (if provided): rendered from JSON
- Tags displayed
- "Submit Solution" button:
  - If visitor: "Sign in to submit" → `/login?redirect=/challenges/[id]/submit`
  - If member: navigates to `/challenges/[id]/submit`
  - If challenge is closed: button disabled "Submissions closed"
- Per-challenge leaderboard: table showing rank, user avatar+name, score, time bonus, total, submitted time
- Real-time updated via Supabase Realtime subscription
- User's own submission history (if authenticated): list of past submissions with timestamp, status, score (if graded), feedback (if provided)

Submit Solution Page (`/challenges/[id]/submit`):

- Auth required. Redirects to login if not authenticated.
- Challenge title and deadline displayed for context
- Form fields:
  - GitHub Repository URL (required): validated with regex for valid GitHub URL format
  - Live Hosted URL (optional): validated for valid URL format
- Duplicate check: before submission, query if this user already submitted to this challenge in the last 60 seconds. If yes, show warning "You submitted recently. Are you sure you want to submit again?"
- On submit: Server Action creates submission row → success toast → redirect to challenge detail page
- After submission: a notification is NOT created (user can see their submission in history). Notification only when admin grades it.

Admin Challenge Management:

- Create: form with title, rich text editor for description and problem statement, difficulty select, category select, start date picker, deadline picker, max score, max time bonus, scoring rules JSON builder, tags
- Edit: same form, pre-filled
- Close: set status to 'closed' (manual, not automatic on deadline)
- Soft delete: confirmation dialog

Admin Submission Review (`/admin/submissions`):

- Filter by challenge (dropdown)
- Table: user name, challenge title, submitted_at, GitHub URL (clickable), live URL (clickable), current status, current score
- Click a row → opens review panel:
  - View GitHub and live URLs (external links)
  - Score input (number, 0 to challenge max_score)
  - Time bonus: auto-calculated from formula `max(0, (deadline - submitted_at) / (deadline - start_date)) * max_time_bonus`, displayed to admin, editable
  - Feedback text area
  - Status dropdown: submitted → under_review → graded
  - "Save Review" button → Server Action updates submission → triggers total_score recalculation → creates notification for the student ("Your submission for [challenge] has been graded")
  - Writes to admin_audit_log

  New: Easy = Success (#00FF88), Medium = Warning (#FFB000), Hard = Error (#FF3366)

8.6 Projects Showcase

URL: `/projects`

Layout:

- Page title: "Projects"
- Filter bar: Tech Stack (multi-select from available tags), Year, Domain, Team Size range
- Search bar: searches project titles
- Featured projects pinned at the top of the grid with a "Featured" badge (#FCF604 on dark)
- Grid of project cards

Project Card:

- Thumbnail (16:9)
- Title
- Description (first 100 characters, truncated)
- Tech stack tags (horizontal scrollable)
- Team member names
- Likes count (heart icon + number)
- Click → navigate to `/projects/[id]`

Project Detail Page (`/projects/[id]`):

- Title, tech stack tags, team members, domain, year
- Screenshots carousel: swipeable on mobile, arrow buttons on desktop, dots indicator
- Full description: rich text rendered with DOMPurify + prose
- Links: GitHub repo, live demo, documentation (each opens in new tab)
- Like button:
  - If visitor: "Sign in to like" tooltip on hover
  - If member (not liked): heart outline → click → Server Action → filled heart + count increments
  - If member (already liked): filled heart → click → Server Action → outline heart + count decrements
  - Optimistic update via TanStack Query mutation

Admin Project Management:

- Create: form with title, rich text editor for description, thumbnail upload, tech stack tags input, team member names input, links, domain, year, team size, screenshots upload (multi-file)
- Edit: same form, pre-filled
- Feature/Unfeature: toggle switch
- Archive: toggle switch
- Soft delete: confirmation dialog

8.7 Media Gallery

URL: `/gallery`

Layout:

- Page title: "Gallery"
- Grid of album cards, sorted by event_date descending

Album Card:

- Cover image (16:9 or square, depending on design)
- Album title
- Event date
- Image count badge
- Click → navigate to `/gallery/[albumId]`

Album Detail Page (`/gallery/[albumId]`):

- Album title, event name, date, description
- Masonry grid layout of images (CSS columns or a masonry library)
- All images lazy-loaded (Next.js `<Image>` with `loading="lazy"`)
- Click any image → Lightbox opens:
  - Full-screen overlay with dark background
  - Current image centered, scaled to fit viewport
  - Left/Right arrow buttons for navigation (keyboard: ArrowLeft, ArrowRight)
  - Close button (keyboard: Escape)
  - Swipe left/right on mobile
  - Image counter: "3 of 24"
  - Caption displayed below image if available
  - Focus trapped within lightbox

Admin Gallery Management (`/admin/gallery`):

- List all albums with edit/delete options
- Create album: title, event date, description, event link (dropdown of events), cover image upload
- Album image management (`/admin/gallery/[albumId]`):
  - View all images in grid
  - Upload new images:
    - Single: click upload button, select file
    - Bulk: click upload button, select multiple files
    - Client-side validation: .jpg/.jpeg/.png/.webp only, max 5MB per file
    - Upload flow:
      1. Files uploaded in parallel via `Promise.allSettled()`
      2. Progress shown: "Uploaded 7 of 12 images"
      3. Failed uploads shown with per-file error
      4. Successfully uploaded URLs collected
      5. Batch insert into `gallery_images`
      6. If batch insert fails: retry once. If retry fails: delete all uploaded files from Storage, show error.
    - After successful upload: images appear in the grid
  - Reorder images: change display_order (drag-and-drop or manual number input)
  - Delete image: confirmation dialog → delete from database AND from Supabase Storage

- Set cover image: click an image to set it as album cover

8.8 Global Leaderboard & Analytics

URL: `/leaderboard`

Layout:

- Page title: "Leaderboard"
- Client Component (real-time updates, no SSR cache)
- Table columns: Rank, Avatar, Name, Score, Rank Badge, Challenges Completed
- Top 3 highlighted with special styling (gold/silver/bronze accents)
- Top 100 loaded on initial fetch
- Supabase Realtime subscription on `profiles` table for `total_score` changes

Rank Badges (computed in application, NOT stored in database):

Badge	Score Range	Display Color
Newbie	0–99	**SHADOW (#A39BBA)**
Contributor	100–499	Glitch (#00F0FF)
Expert	500–999	Warning (#FFB000)
Legend	1000+	Hologram (#E6007E)

Personal Analytics (on `/dashboard`):

- Score over time: line chart (Recharts) showing cumulative score after each graded submission
- Participation rate: number of challenges attempted / total challenges
- Win rate: number of first-place finishes / total submissions
- Activity heatmap: GitHub-style grid of colored cells, 365 days, derived from the UNION query across submissions, registrations, likes, and blog posts. Intensity based on `actions_count` per day.

Admin Analytics (on `/admin` dashboard):

- Total registered users (counter)

- New signups over time (line chart, last 30 days)
- Total submissions over time (line chart, last 30 days)
- Event registrations over time (line chart, last 30 days)
- Active users (users who performed any action in last 7 days)
- Top challenges by submission count (bar chart)

8.9 In-App Notifications

Bell Icon (in Navigation):

- Shows unread count as a badge
- Click opens dropdown panel (Radix Popover)

Notification Types:

Type	Trigger	Message Template	Link
submission_graded	Admin grades a submission	"Your submission for {challenge_title} has been graded. Score: {score}"	`/challenges/{id}`
badge_earned	Badge award logic triggers	"You earned the {badge_name} badge!"	`/dashboard`
event_reminder	(Post-hackathon: cron job. Hackathon: manual admin action via announcement)	"Reminder: {event_title} starts tomorrow!"	`/events/{id}`
challenge_deadline	(Same as above)	"Reminder: {challenge_title} deadline is in 24 hours!"	`/challenges/{id}`

Notification Dropdown:

- Lists last 20 notifications, sorted by created_at descending
- Each: icon (color-coded by type), title, message, relative time ("2 hours ago")

- Unread: white background. Read: grey background.
- Click notification: mark as read + navigate to link
- "Mark all as read" button
- Announcements appear at the top of the dropdown (from the UNION query with `last_dismissed_announcements_at`). "Dismiss announcements" button updates the profile timestamp.

8.10 Badges & Achievement System

Badge Definitions (seeded in database):

Badge Name	Criteria Type	Criteria Value	Trigger Point
First Submission	first_submission	NULL	After first submission is created
First Win	first_win	NULL	After first submission is graded with highest score for that challenge
Challenge Veteran	challenges_completed	5	After 5th unique challenge with a graded submission
Event Regular	events_attended	3	After admin marks 3rd event attendance
Legend	total_score	1000	After total_score crosses 1000

Badge Award Logic:

- Runs server-side in the relevant Server Action
- After a submission is graded: check `first_submission`, `first_win`, `challenges_completed`, `total_score` badges
- After attendance is marked: check `events_attended` badge
- Each check: query if user already has the badge (user_badges). If not, and criteria is met, insert into user_badges and create a notification.

- This is straightforward if/else logic, not a complex rules engine.

Badge Display:

- On public profile page: grid of earned badges with icon and name
- On dashboard: same grid with earned dates
- On leaderboard: user's highest badge displayed next to their name

8.11 Streaks

Login Streak Logic:

- On every login (detected in middleware or auth listener):
  1. Read `profiles.last_active_date`
  2. Get today's date in UTC
  3. If `last_active_date` is yesterday: increment `current_streak` by 1
  4. If `last_active_date` is today: do nothing (already counted)
  5. If `last_active_date` is older than yesterday: reset `current_streak` to 1
  6. If `current_streak > longest_streak`: update `longest_streak`
  7. Set `last_active_date` to today
- All comparisons in UTC dates.

Display:

- Dashboard: "Current Streak: X days " and "Longest Streak: Y days"
- If streak is 7+, show a flame animation next to the number

8.12 Blog System

URL: `/blog`

Blog Listing Page:

- Grid of blog post cards
- Each card: cover image, title, author avatar + name, published date, tags
- Click → `/blog/[id]`
- Search: by title (server-side ILIKE)
- Filter: by tag

Blog Post Detail Page:

- Cover image (full width)
- Title (h1)
- Author info: avatar, name, link to profile
- Published date
- Tags

- Content: rich text rendered with DOMPurify + Tailwind prose
- No comments (out of scope for hackathon)

Blog Creation (`/blog/new`):

- Auth required (any member)
- Tiptap rich text editor (loaded dynamically with `ssr: false`)
- Title input
- Cover image upload
- Tags input (comma-separated or tag picker)
- Status toggle: Draft / Published
- "Save" button → Server Action creates/updates blog post
- If status = 'published' and `published_at` is null, set `published_at = now()`

Blog Edit (`/blog/edit/[id]`):

- Same form, pre-filled
- Only the author or an admin can edit
- Auth guard checks ownership or admin role

8.13 Certificate Generation

Flow:

1. An event must have status = 'finished'
2. Admin must have marked attendees (event_registrations.status = 'attended')
3. On the event detail page (for authenticated attendees) or on the dashboard, a "Download Certificate" button appears if:
   - The event is finished AND
   - The user's registration status is 'attended'
4. User clicks "Download Certificate"
5. Client-side: a hidden HTML certificate template is rendered with:
   - Club logo
   - "Certificate of Participation" (or "Certificate of Achievement" for winners, determined by event results JSON)
   - User's full_name
   - Event title
   - Event date
   - A unique certificate ID (deterministic: hash of event_id + user_id)
6. `html2canvas` renders the HTML element to a canvas
7. `jsPDF` creates an A4 landscape PDF from the canvas
8. PDF is downloaded to the user's device
9. A row is inserted/updated in `certificates` table with `downloaded_at = now()`
10. Total time: under 2 seconds for one certificate

Certificate Template Design:

Property	Value
Size	A4 Landscape (297mm × 210mm)
Background	Void (`#0A0514`)
Border	2px Hologram (`#E6007E`) with 16px margin from edge, rounded-lg
Top center	Boot Force logo (white version)
Main heading	"CERTIFICATE OF PARTICIPATION" or "CERTIFICATE OF ACHIEVEMENT" — Space Grotesk 700, Light (`#F2F0FF`)
Recipient line	"This is awarded to" — Inter 400, #D9E2EC
Name	User's full_name — Space Grotesk 700, Glitch (`#00F0FF`), large (36px)
Event line	"for participating in {event_title}" — Inter 400, #FFFFFF
Date	Event date — Inter 400, #D9E2EC
Certificate ID	Bottom right corner — JetBrains Mono 400, #A0AEC0, small
Signature area	Bottom center — "Boot Force Team" with a line above

8.14 Alumni Network

Part of the Team Section, not a separate page.

- Toggle switch on `/team` page: "Show Alumni"
- When ON: grid filters to `team_members WHERE is_alumni = true`
- Alumni cards show additional fields: `alumni_current_role`, `alumni_current_company`

- Visual distinction: a subtle "Alumni — Class of {batch_year}" badge on the card
- When OFF: grid shows active members only (is_alumni = false)

8.15 Admin Panel

URL: /admin

Dashboard (/admin):

- Welcome message with admin name
- Quick stats cards: Total Users, Total Events, Total Challenges, Total Submissions, Active Users (7 days)
- Charts (Recharts):
  - New signups over time (line chart, 30 days)
  - Submissions over time (line chart, 30 days)
  - Event registrations over time (line chart, 30 days)
- Recent admin actions (from audit log, last 10)

User Management (/admin/users):

- Table: avatar, name, email, role, total_score, created_at
- Search by name or email
- Actions per user:
  - "Promote to Admin" (confirmation dialog) → Server Action updates profiles.role → audit log
  - "Ban User" (confirmation dialog) → Server Action (implementation: set a is_banned column on profiles, middleware checks this on login) → NOTE: is_banned column not yet in schema — ADD IT
  - "View Profile" → navigates to /profile/[id]

SCHEMA ADDITION: Add is_banned BOOLEAN DEFAULT false to profiles table. Auth middleware checks: if is_banned = true, sign out the user and redirect to /login with a message.

Announcement Management (/admin/announcements):

- List of all announcements with active/inactive toggle
- Create: title + content form
- Toggle active/inactive → Server Action updates is_active
- Delete: confirmation dialog → hard delete

Audit Log (/admin/audit-log):

- Table: admin name, action, entity type, entity ID, timestamp, details
- Filters: by action type, by date range
- Pagination: server-side, 50 per page, cursor-based (append-only table)
- Read-only — no edit or delete

SECTION 9: SEED DATA STRATEGY

9.1 Purpose

Seed data populates the database with realistic-looking content so that every page has something to display from Phase 2 onward. Without seed data, the homepage shows empty carousels, the team page is blank, and the demo is unpresentable.

9.2 Seed Script

A script at `scripts/seed.ts` that runs via `npx tsx scripts/seed.ts`.

Behavior: Wipes all data from all tables (in correct order to respect foreign keys) and reinserts fresh seed data. NOT idempotent — it's destructive. Documented with a warning comment.

Deletion order (respects foreign keys):

1. admin_audit_log
2. certificates
3. user_badges
4. notifications
5. project_likes
6. gallery_images
7. gallery_albums
8. blog_posts
9. submissions
10. event_registrations
11. announcements
12. badges
13. team_members
14. projects
15. challenges
16. events
17. profiles (except admin accounts)

Seed data:

Entity	Count	Details
Admin account	1	`admin@bootforce.com`, role: admin. Pre-seeded separately (not wiped).
Student account	1	`student@bootforce.com`, role: member. Used for demo.
Additional members	3	Varied tech stacks, bios, avatars from randomuser.me
Team members	6	2 leads, 2 core_team, 2 members. Different domains and batches. Photos from randomuser.me.
Alumni	1	is_alumni=true, with current role and company
Events	4	2 upcoming (1 workshop 3 days from now, 1 hackathon 7 days from now), 1 ongoing, 1 finished (with results JSON)
Event registrations	5	Student account registered for 2 upcoming events and 1 finished event (status: attended)
Challenges	4	1 easy/active, 1 medium/active, 1 hard/active, 1 medium/closed
Submissions	5	From student account: 2 for the closed challenge (1

		graded with score 85, 1 graded with score 72), 1 for an active challenge (submitted, not graded), 2 from other members (graded) for leaderboard variety
Projects	4	Different tech stacks. 1 is_featured=true. Thumbnails from picsum.photos.
Project likes	3	From various member accounts
Gallery albums	1	Linked to the finished event. Cover image from picsum.photos.
Gallery images	8	In the album. Images from picsum.photos with varied sizes.
Badges	5	First Submission, First Win, Challenge Veteran (5), Event Regular (3), Legend (1000)
User badges	2	Student account has "First Submission" badge
Blog posts	2	1 published (by student account), 1 draft (by student account)
Announcements	1	Active, with a welcome message

Placeholder image sources:

- Faces: `https://randomuser.me/api/portraits/men/{n}.jpg` and `.../women/{n}.jpg`
- Photos: `https://picsum.photos/seed/{keyword}/{width}/{height}`

9.3 When to Run

- After Phase 1 (database setup) to verify tables work
- At the start of Phase 2 to populate homepage
- Before the final demo to ensure clean, realistic data
- Anytime data gets messy during development

SECTION 10: DEPLOYMENT & ENVIRONMENT CONFIGURATION

10.1 Supabase Project Setup

1. Create a new Supabase project at `supabase.com`
2. Note the following values from Project Settings → API:
   - Project URL (`NEXT_PUBLIC_SUPABASE_URL`)
   - Anon/Public key (`NEXT_PUBLIC_SUPABASE_ANON_KEY`)
   - Service Role key (`SUPABASE_SERVICE_ROLE_KEY`) — NEVER expose this to the client
3. Configure Auth → Providers:
   - Email: enabled (default)
   - Google: disabled initially, enabled in Phase 7B (requires Google Cloud Console OAuth app)
   - GitHub: disabled initially, enabled in Phase 7B (requires GitHub Developer app)
4. Configure Auth → URL Configuration:
   - Site URL: `https://your-vercel-url.vercel.app`
   - Redirect URLs: `https://your-vercel-url.vercel.app/auth/callback`
5. Create Storage buckets:
   - `avatars` — public read, authenticated write, 2MB max file size, image types only
   - `banners` — public read, admin write, 5MB max, image types only
   - `gallery` — public read, admin write, 5MB max, image types only
   - `projects` — public read, admin write, 5MB max, image types only
   - `certificates` — authenticated read (own files only), service role write

10.2 Environment Variables

`.env.local` (local development):

```text
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co

NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
```

`SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi...`

Vercel Environment Variables (production):

Same three variables, configured in Vercel dashboard → Project → Settings → Environment Variables.

Rules:

- `NEXT_PUBLIC_*` variables are exposed to the browser. Safe because RLS protects the database.
- `SUPABASE_SERVICE_ROLE_KEY` is server-only. NEVER prefixed with `NEXT_PUBLIC_`. Used only in Server Actions and API routes.
- No other environment variables needed for the hackathon.

10.3 Vercel Deployment

1. Push code to GitHub
2. Connect GitHub repo to Vercel
3. Vercel auto-detects Next.js framework
4. Set environment variables in Vercel dashboard
5. Deploy
6. Every push to `main` triggers an auto-deployment
7. HTTPS is automatic on all Vercel deployments
8. Custom domain optional — `.vercel.app` subdomain is fine for hackathon

10.4 File Storage Policies

Supabase Storage bucket policies:

`avatars` bucket:

- Public access for read (anyone can view avatars)
- Authenticated users can upload to their own folder (`avatars/{user_id}/`)
- Max file size: 2MB
- Allowed MIME types: image/jpeg, image/png, image/webp

`banners` bucket:

- Public access for read
- Only admins can upload (enforced by RLS policy on the bucket or by only calling upload from admin Server Actions)
- Max file size: 5MB
- Allowed MIME types: image/jpeg, image/png, image/webp

`gallery` bucket:

- Same as banners

`projects` bucket:

- Same as banners

File replacement cleanup:

When updating an image (e.g., changing an event banner), the Server Action:

1. Uploads the new file
2. If upload succeeds, updates the database row with the new URL
3. Deletes the old file from Storage
4. If old file deletion fails, logs the orphan path to audit_log (non-blocking)

SECTION 11: GIT WORKFLOW

11.1 Branch Strategy

Single `main` branch. No feature branches.

Rationale: One developer, supervised build, sequential phases. Feature branches add merge overhead with zero benefit.

11.2 Commit Convention

Format: `phase-X: description in lowercase`

Examples:

```text
phase-1: initialize next.js project with typescript and tailwind

phase-1: create supabase tables and rls policies

phase-1: configure supabase auth with ssr cookie flow

phase-2: build hero section with animated stats counters

phase-2: implement club pulse 2d visualization

phase-3: build team section with filter and modal

phase-4: implement event detail page with registration flow

phase-5: add realtime leaderboard subscription

phase-6: build gallery lightbox with keyboard navigation

phase-7a: create admin analytics dashboard with recharts
```

`phase-7b: implement badge award logic and profile display`

11.3 Commits Per Phase

Granular. Each logical unit of work is a separate commit. NOT one squash commit per phase. This demonstrates "meaningful commit history" per the evaluation criteria.

Typical phase has 8-15 commits.

11.4 Deploy Cadence

Every push to `main` auto-deploys to Vercel. The live URL is always the latest commit. After Phase 1, there's a working (if skeletal) live site. It gets progressively better with each phase.

SECTION 12: THE DEMO SCRIPT

12.1 Pre-Demo Preparation

1. Run seed script to ensure clean, realistic data
2. Delete any garbage test content (blog posts, submissions)
3. Verify admin account login works: `admin@bootforce.com`
4. Verify student's pre-seeded submission exists and is ungraded (or re-seed one)
5. Open the live URL in two browser tabs:
   - Tab 1: The demo walkthrough tab
   - Tab 2: The leaderboard page (for the real-time moment)
6. Test the full demo flow once privately before presenting
7. Have mobile device ready (or browser DevTools responsive mode) for the mobile showcase

12.2 The Script — 3 Minutes

[0:00 – 0:30] The Hook — Homepage

Open Tab 1 to the homepage. The cinematic hero loads with the Club Pulse visualization.

"This is Boot Force — the official hub for our technical club at CMR University. What you're seeing is a live visualization of our club's network — every node represents a real member, event, or project in our system."

Scroll down slowly. Stats counters animate. Upcoming events strip slides into view. Featured projects carousel rotates.

"Everything here is live data. These stats, these events, these projects — all pulled from our database in real-time."

[0:30 – 1:00] The Visitor Experience

Click an upcoming event card. Show the detail page — countdown timer ticking, description, schedule, speakers.

Click "Register" → the "Sign in to register" prompt appears.

"Any visitor can explore the entire platform without logging in. The moment they want to participate, we guide them to create an account."

Navigate to Team page. Show the hierarchy — Leads, Core Team, Members. Click a filter. Click a card to open the modal.

Navigate to Gallery. Open an album. Click a photo — lightbox opens, full-screen.

"Every event is documented. Every member is showcased."

[1:00 – 1:45] The Real-Time Moment

"Now let me show you the competitive engine."

Arrange Tab 1 and Tab 2 side by side (or switch between them clearly).

Log in as admin in Tab 1. Navigate to Admin → Submissions.

Show Tab 2: the leaderboard page, visible.

"I'm going to grade a submission right now. Watch the leaderboard."

In Tab 1: open a pending submission. Enter a score. Click "Save Review."

Switch to Tab 2: the leaderboard updates — score changes, rank potentially shifts.

"No refresh. That's a live WebSocket connection. The leaderboard updates the instant I grade a submission."

[1:45 – 2:30] The Admin Experience

Stay logged in as admin.

Navigate to Admin → Events → Create New Event. Fill in a title, pick a date, write a quick description, upload a banner image, select a category. Click "Publish."

Switch to the public Events page. The new event appears.

"One admin panel controls everything — events, challenges, projects, gallery, users."

Quick flash through: Admin dashboard (analytics charts), User management page, Audit log.

"Every admin action is logged. Full transparency."

[2:30 – 3:00] The Close

Toggle to mobile view (responsive mode or real phone).

Show the homepage on mobile — navigation collapses to hamburger, layout adapts, visualization still renders.

Scroll through events on mobile. Open the lightbox on mobile.

"Fully responsive. Works on every device."

Return to desktop. Show the homepage hero one more time.

"Boot Force isn't just a website. It's the club's digital identity. One platform for team, events, challenges, projects, and memories. Built for every member — from curious freshers to experienced seniors."

SECTION 13: PHASE BREAKDOWN

Overview

The build is divided into **8 phases** executed sequentially. Each phase has a clear scope, a definition of done checklist, estimated time, and notes on what can be automated via AI agent and what requires manual supervision.

The original 7 phases have been restructured: UI/UX design is now Phase 1, and the old Phase 7 is split into 7A and 7B, making 8 total phases.

Phase	Name	Focus	Estimated Time
1	UI/UX Design System & Component Library	Design foundation	3–4 hours
2	Foundation & Infrastructure	Database, auth, project setup	3–4 hours
3	Homepage & Global Layout	First visual impression	3–4 hours
4	Team Section & User Profiles	People pages	2–3 hours
5	Events System	Discovery and participation	3–4 hours
6	Challenges, Submissions & Real-Time		

Leaderboard | Competitive engine | 3–4 hours |
7	Projects Showcase & Media Gallery	Visual proof	2–3 hours
8A	Admin Panel, Notifications, Blog	Admin completion	2–3 hours
8B	Badges, Streaks, Certificates, Final Polish	Everything else + demo prep	2–3 hours

Total estimated: 23–32 hours

Priority rule: If time runs short, phases are cut from the bottom. Phases 1–6 represent the highest-scoring features. Phases 7–8B are progressively less critical to the hackathon score.

PHASE 1: UI/UX DESIGN SYSTEM & COMPONENT LIBRARY

Goal

Establish the complete visual foundation before writing any feature code. Every color, font, spacing token, animation, and reusable component is built here. No feature pages are built in this phase — only the design primitives and component library that every subsequent phase will use.

Why This Is Phase 1

UI/UX is 25% of the hackathon score — the single highest-weighted category. If the design system is inconsistent, every page built in later phases will look inconsistent. Fixing design inconsistencies retroactively across 15+ pages is far more expensive than getting it right upfront.

Scope

1.1 Tailwind Configuration

- Configure `tailwind.config.ts` with the complete color palette from Section 6.1
- Define custom color names that map to hex values:
  - `navy: '#0B3A63'`
  - `steel: '#1E5A8A'`
  - `blue-grey: '#D9E2EC'`
  - `brand-red: '#FC0204'`
  - `crimson: '#E63946'`
  - `maroon: '#640204'`
  - `salmon: '#FCA2A4'`
  - `bright-yellow: '#FCF604'`
  - `golden: '#F5A623'`
  - `olive: '#8C8604'`
  - `charcoal: '#222222'`
  - `muted: '#6C757D'`
  - `cool-grey: '#A0AEC0'`
- Configure font families: Space Grotesk (display + headings), Inter (body), JetBrains Mono (data/code)
- Configure the type scale as custom fontSize entries matching Section 6.2
- Configure animation durations and easings as custom Tailwind theme extensions
- Install and configure `@tailwindcss/typography` for prose styling
- Add the `darkMode: 'class'` setting (even though we're dark-first, this allows conditional dark styling)

1.2 Global Styles

- CSS file with font imports (Google Fonts: Space Grotesk, Inter, JetBrains Mono)
- Base body styles: default background color, default text color, font smoothing
- Scrollbar styling (subtle, dark-themed)
- Selection color (brand accent)
- Focus ring styles (visible, accessible, brand-colored)

1.3 shadcn/ui Installation and Customization

- Initialize shadcn/ui in the project
- Install the following components, customized with our brand colors:
  - **Button** — variants: primary (red), secondary (outlined), ghost, destructive
  - **Input** — styled with focus ring matching brand

    - **Textarea** — same styling as Input
    - **Select** — dropdown with brand styling
    - **Dialog / Modal** — overlay with navy background at 80% opacity, white card surface
    - **Dropdown Menu** — for navigation user menu, admin actions
    - **Tabs** — for Events page (Upcoming/Past), admin sections
    - **Tooltip** — for icon-only buttons and disabled states
    - **Popover** — for notification bell dropdown
    - **Toast** — success (green), error (red), info (blue) variants
    - **Badge** — for difficulty labels, category tags, status indicators
    - **Card** — base card component matching Section 6.4 tokens
    - **Avatar** — circular image with fallback initials
    - **Skeleton** — loading placeholders matching page layouts
    - **Switch** — for toggles (alumni, publish/unpublish)
    - **Accordion** — for event FAQs
    - **Command** — for search inputs (if needed)
    - **Separator** — horizontal rule with brand styling
    - **ScrollArea** — for horizontal scrollable sections
    - **Sheet** — for mobile navigation drawer
- Every component must be tested with both light surface (#FFFFFF) and dark surface (#0B3A63, #1E5A8A) backgrounds to verify contrast

1.4 Custom Reusable Components

These are NOT shadcn/ui components. They are custom components built with Tailwind and Framer Motion that will be reused across multiple pages:

- **SectionHeader** — a consistent section title component used across all pages. Props: title, subtitle (optional), alignment (left/center). Uses Space Grotesk font.
- **StatCounter** — animated number counter that counts from 0 to target value on scroll. Uses JetBrains Mono. Props: value, label, duration. Used on homepage stats bar.
- **CountdownTimer** — live ticking countdown showing Days:Hours:Minutes:Seconds. Props: targetDate. Uses JetBrains Mono. Used on event cards and detail pages.
- **TagList** — horizontal scrollable row of tech stack or skill tags. Props: tags (string array), variant (light/dark). Used on team cards, project cards, challenge cards.
- **PageTransition** — Framer Motion wrapper that applies the crossfade animation to page content. Wraps every page's main content.
- **ScrollReveal** — Framer Motion wrapper that applies the fade-up scroll-triggered animation. Wraps sections and cards that should animate on scroll.
- **StaggerContainer** and **StaggerItem** — Framer Motion components for staggered list animations. Used on card grids.
- **EmptyState** — displayed when a listing has no items. Props: icon, message, actionLabel, actionLink. Consistent empty state across all pages.
- **LoadingGrid** — skeleton grid matching the card layout. Props: count (number of skeleton cards). Used in loading.tsx files.

- **RankBadge** — displays the user's rank badge (Newbie/Contributor/Expert/Legend) with appropriate color. Props: totalScore. Computes badge from score thresholds.

1.5 Layout Components

- **RootLayout** — the outermost layout wrapper applied to every page. Contains:
  - Font loading (Space Grotesk, Inter, JetBrains Mono)
  - Metadata defaults (title, description, Open Graph)
  - Auth Context Provider (wraps the entire app)
  - TanStack Query Provider (wraps the entire app)
  - Toast container (renders toasts at the bottom of viewport)
  - Skip-to-content link (accessibility)
- **Navbar** — full navigation bar component (desktop and mobile versions). Built in this phase as a shell. Auth-aware behavior (sign in button vs. avatar dropdown) is wired in Phase 2 when auth is implemented. Notification bell is a visual placeholder until Phase 8A.
- **Footer** — complete footer component with all links and social icons.
- **AdminLayout** — layout wrapper for all `/admin/*` routes. Sidebar navigation with links to all admin sub-pages. Content area on the right.

1.6 Framer Motion Setup

- Create an `AnimatePresence` wrapper in the root layout for page transitions
- Define motion variants as constants in a shared file:
  - `fadeUp` — the standard scroll-triggered entrance
  - `fadeIn` — simple opacity fade
  - `staggerContainer` — parent variant with stagger children delay
  - `staggerItem` — child variant for staggered lists
  - `scaleOnHover` — card hover effect (scale 1.02, shadow-xl)
  - `pageTransition` — crossfade for page changes
- Test: create a temporary test page with all animation variants to verify they look correct

1.7 Dark Theme Verification

- Create a temporary `/design-system` page (removed before final build) that displays:
  - Every color on every possible background (navy, steel, white, light grey)
  - Every text color on every background — verifying the contrast rules from Section 6.1
  - Every button variant on light and dark backgrounds
  - Every card variant
  - Typography samples at all sizes
  - Animation samples
- This page is the design system's visual test. Screenshot it for reference during later phases.

What Can Be Automated (AI Agent)

- Tailwind config generation from the color/font/spacing specs

- shadcn/ui component installation and base customization
- Boilerplate for custom components (StatCounter, CountdownTimer, etc.)
- Layout component shells (Navbar, Footer, AdminLayout)
- Framer Motion variant definitions

What Requires Manual Supervision

- Verifying color contrast on the design system test page (visual check)
- Fine-tuning shadcn/ui component styling to match the cinematic aesthetic
- Adjusting animation timing and easing until they "feel" right
- Ensuring mobile navigation (Sheet/drawer) works correctly on touch
- Design decisions that aren't specified here (e.g., exact icon choices, card border treatments, gradient overlays on hero sections)

Definition of Done

- Tailwind config has all custom colors, fonts, spacing, and animation tokens
- All Google Fonts load correctly
- All shadcn/ui components listed above are installed and customized
- Every component renders correctly on both light (#FFFFFF, #F4F6F8) and dark (#0B3A63, #1E5A8A) backgrounds
- All custom reusable components (StatCounter, CountdownTimer, TagList, etc.) render correctly in isolation
- Navbar renders on desktop (full links) and mobile (hamburger → sheet/drawer)
- Footer renders with all links and social icons
- AdminLayout renders with sidebar navigation
- Page transition animation works when navigating between any two pages
- Scroll-triggered fade-up animation works on a test section
- Stagger animation works on a test grid of cards
- The `/design-system` test page displays all colors, typography, components, and animations
- No contrast violations visible on the test page
- No console errors or warnings
- Committed to GitHub with granular commits

PHASE 2: FOUNDATION & INFRASTRUCTURE

Goal

Set up the complete backend infrastructure: database tables, triggers, views, RLS policies, authentication flow, Supabase client utilities, middleware, and project scaffolding. After this phase, the app has working auth, a protected route structure, and a seeded database. Visually, it's just the layout shell from Phase 1 with login/signup pages.

Scope

2.1 Project Initialization

- Initialize Next.js 14 project with App Router and TypeScript (if not already done in Phase 1)
- Configure `tsconfig.json` with path aliases (`@/components`, `@/lib`, `@/types`, etc.)
- Set up project folder structure:

```text
src/
├── app/
│   ├── (public)/        # Public routes group
│   │   ├── page.tsx      # Homepage
│   │   ├── team/
│   │   ├── events/
│   │   ├── challenges/
│   │   ├── projects/
│   │   ├── gallery/
│   │   ├── leaderboard/
│   │   ├── blog/
│   │   └── profile/[id]/
│   ├── (auth)/          # Auth routes group
│   │   ├── login/
│   │   └── signup/
│   ├── (member)/        # Member-protected routes
│   │   ├── dashboard/
│   │   ├── profile/settings/
│   │   └── blog/new/
│   ├── (admin)/         # Admin-protected routes
│   │   └── admin/
│   ├── auth/callback/     # OAuth callback handler
```

```
│   ├── layout.tsx        # Root layout
│   ├── loading.tsx       # Global loading
│   ├── error.tsx         # Global error
│   └── not-found.tsx     # 404 page
├── components/
│   ├── ui/               # shadcn/ui components
│   ├── layout/           # Navbar, Footer, AdminLayout
│   ├── shared/           # Custom reusable components
│   └── features/         # Feature-specific components (by module)
├── lib/
│   ├── supabase/
│   │   ├── client.ts     # Browser Supabase client
│   │   ├── server.ts     # Server Supabase client
│   │   └── admin.ts      # Service role client (server only)
│   ├── validators/       # Zod schemas
│   ├── utils.ts          # Shared utilities (cn, formatDate, etc.)
│   └── constants.ts      # App-wide constants (rank thresholds, etc.)
├── hooks/                # Custom React hooks
├── types/                # TypeScript type definitions
├── styles/               # Global CSS
└── scripts/
    └── seed.ts           # Seed data script
```

2.2 Supabase Client Setup

- Create browser client (`lib/supabase/client.ts`) using `createBrowserClient` from `@supabase/ssr`
- Create server client (`lib/supabase/server.ts`) using `createServerClient` from `@supabase/ssr` with cookie handling
- Create admin client (`lib/supabase/admin.ts`) using `createClient` with the service role key — used only in Server Actions that need to bypass RLS

- Verify: browser client can query public data, server client can read auth session from cookies, admin client can perform privileged operations

2.3 Authentication Implementation

- Build `/login` page:
  - Email input, password input, "Sign In" button
  - Link to `/signup` (carries `redirect` param if present)
  - Placeholder OAuth buttons (disabled, with "Coming soon" tooltip)
  - Error display for invalid credentials
  - On success: redirect to `redirect` query param or `/dashboard`
  - Styled with the design system from Phase 1
- Build `/signup` page:
  - Full name input, email input, password input, confirm password input, "Create Account" button
  - Link to `/login` (carries `redirect` param)
  - Placeholder OAuth buttons (disabled)
  - Zod validation: email format, password min 8 chars, passwords match
  - On success: redirect to `redirect` query param or `/dashboard`
  - Styled with the design system
- Build `/auth/callback` route:
  - Handles OAuth callback (for Phase 8B when OAuth is activated)
  - Exchanges code for session
  - Redirects to stored redirect URL from localStorage, validated to start with `/`
- Build `middleware.ts`:
  - Runs on every request matching protected route patterns
  - Creates Supabase server client, calls `getUser()` to refresh session
  - For member routes: if no session, redirect to `/login?redirect={current_path}`
  - For admin routes: if no session, redirect to `/login`. If session but role != 'admin', redirect to `/`
  - If user `is_banned = true`: sign out, redirect to `/login` with banned message
  - Does NOT run on: `/_next/*`, `/favicon.ico`, static assets, public routes

2.4 Auth Context Provider

- Create `AuthProvider` component wrapping the entire app
- On mount: calls `supabase.auth.getSession()` to get initial session
- Listens to `supabase.auth.onAuthStateChange()` for login/logout/token refresh events
- Provides via React Context: `user` (current user object or null), `profile` (profiles row with role), `isLoading` (true during initial session check)
- All components can use `useAuth()` hook to access current user and role
- Profile data (including role) is fetched once on auth state change and cached in context

2.5 Database Setup

- Create ALL tables defined in Section 4.1 in the Supabase SQL editor
- Create ALL triggers defined in Section 4.2

- Create ALL views defined in Section 4.3 with `security_invoker = true`
- Grant SELECT on all views to `anon` and `authenticated` roles
- Enable RLS on every table
- Create ALL RLS policies defined in Section 4.4
- Create the `is_banned` column on profiles (added in Section 8.15)

2.6 TypeScript Types

- Define TypeScript types for every database table row:
  - `Profile`, `TeamMember`, `Event`, `EventRegistration`, `Challenge`, `Submission`, `Project`, `ProjectLike`, `GalleryAlbum`, `GalleryImage`, `Notification`, `Announcement`, `Badge`, `UserBadge`, `BlogPost`, `Certificate`, `AuditLogEntry`
- These types mirror the database schema exactly
- Used throughout the app for type-safe queries and components

2.7 Supabase Storage Buckets

- Create all storage buckets as defined in Section 10.1:
  - `avatars`, `banners`, `gallery`, `projects`, `certificates`
- Configure bucket policies (public read where needed, auth restrictions on write)
- Verify: can upload a test image and retrieve its public URL

2.8 Seed Admin Account

- Insert the admin account directly via Supabase SQL or Auth dashboard:
  - Email: `admin@bootforce.com`
  - Password: (chosen, documented privately)
  - After signup, manually set `profiles.role = 'admin'` via SQL

2.9 Seed Script

- Write the seed script as defined in Section 9
- Run it and verify all tables populate correctly
- Verify foreign key relationships are intact
- Verify triggers fire correctly (registration_count, likes_count, total_score update)

2.10 Error and Loading Pages

- Build `error.tsx` (global error boundary): styled with design system, "Something went wrong" message, retry button
- Build `not-found.tsx` (404 page): styled, "Page not found" message, link back to homepage
- Build `loading.tsx` (global loading): full-page skeleton

What Can Be Automated (AI Agent)

- SQL generation for all tables, triggers, views, RLS policies (from the specs in Sections 4.1–4.4)

- TypeScript type definitions (derived directly from table schemas)
- Supabase client utility files (standard boilerplate)
- Middleware boilerplate
- Login/signup page components
- Auth context provider
- Seed script generation
- Error/loading page components
- Project folder structure creation

What Requires Manual Supervision

- Running SQL in Supabase dashboard and verifying it executes without errors
- Testing auth flow end-to-end: signup → profile created → login → session active → logout → session cleared
- Verifying RLS policies by attempting unauthorized access (try to query another user's notifications, try to modify data as a non-admin)
- Verifying triggers fire correctly (register for an event, check that registration_count incremented)
- Testing middleware redirects: access `/dashboard` without login → redirected to `/login?redirect=/dashboard`
- Creating and verifying the admin account
- Running the seed script and spot-checking data

Definition of Done

- Next.js project runs without errors (`npm run dev`)
- All database tables created with correct types and constraints
- All triggers verified: insert a test registration → event.registration_count updates
- All views created with `security_invoker = true` — verified by querying as anon (soft-deleted rows hidden)
- All RLS policies active — verified by attempting unauthorized operations from the browser client (should fail)
- Supabase browser client, server client, and admin client all work
- Email/password signup creates a user AND a profiles row
- Login returns a session. Server Components can read the session from cookies.
- Logout clears the session.
- Auth Context provides user and profile data throughout the app
- Middleware redirects unauthenticated users from `/dashboard` to `/login?redirect=/dashboard`
- Middleware redirects non-admins from `/admin` to `/`
- Login page reads `redirect` param and navigates there after successful login
- Signup page carries `redirect` param and navigates there after account creation
- One admin account seeded and can log in
- Login page has disabled OAuth buttons with "Coming soon" tooltip
- All TypeScript types defined and match database schema
- All storage buckets created with correct policies

- Seed script runs successfully and populates all tables
- Error page, 404 page, and loading page render correctly
- Root layout renders Navbar shell and Footer
- Deployed to Vercel — live URL loads without errors
- Committed to GitHub with granular commits

PHASE 3: HOMEPAGE & GLOBAL LAYOUT

Goal

Build the first thing anyone sees. This phase directly impacts the "first impression" that judges form. The homepage must be cinematic, animated, and populated with real seed data. The navigation bar becomes fully functional. The Club Pulse visualization is attempted here (with the 2-hour fallback trigger).

Scope

3.1 Navigation Bar — Full Implementation

- Wire up auth state: show "Sign In" when unauthenticated, show avatar dropdown when authenticated
- Avatar dropdown: Profile, Dashboard, Settings, Logout. Admin users also see "Admin Panel" link.
- Notification bell: visual only in this phase (icon + static badge). Functionality wired in Phase 8A.
- Mobile hamburger menu: Sheet/drawer with all links, auth-state-aware
- Active link highlighting based on current route
- Navbar should be semi-transparent on the homepage hero (glassmorphism effect) and solid on all other pages

3.2 Hero Section

- Full viewport height
- Deep Navy (#0B3A63) background with subtle gradient overlay
- "BOOT FORCE" in Space Grotesk 700, display size
- Tagline below in Inter 400
- Two CTA buttons with hover animations:
  - "Join Boot Force" → `/signup` (red primary button)
  - "Explore" → smooth scroll to stats section (outlined button)
- Entrance animation: text elements fade up with stagger, buttons fade up with delay
- Club Pulse visualization (see 3.3)

3.3 Club Pulse Visualization

- **Hour 0–2 of this phase: Attempt Plan A (Three.js 3D)**
  - React Three Fiber + drei + d3-force-3d
  - Fetch node data server-side: count of members, events, challenges, projects → pass as props
  - Generate node array: one node per entity with type-based color
  - Generate edge array: connections between related entities

    - Render instanced spheres for nodes, line segments for edges
    - OrbitControls for mouse interaction (rotate, zoom)
    - Custom glow shader on nodes
    - Ambient floating animation
    - Test at 60fps on desktop, 30fps on mobile
- **Decision point at 2 hours into this phase:**
    - If 3D graph looks visually pleasing, interactive, and performs well → keep Plan A
    - If ANY of these fail → immediately pivot to Plan B
- **Plan B (2D SVG with Framer Motion) — the fallback AND mobile version:**
    - Same concept: nodes and edges representing club entities
    - SVG elements positioned using a deterministic layout function (circular/organic arrangement based on entity count)
    - Nodes rendered as colored circles with Framer Motion `animate` for gentle floating
    - Edges as SVG `<line>` elements with low opacity
    - Hover on nodes reveals labels via Tooltip
    - Click navigates to the entity's page
    - No physics simulation — positions are pre-calculated, predictable, fast
    - Works on all devices, all browsers
- **Mobile rule:** On screens < 768px, always show Plan B regardless of whether Plan A succeeded. Three.js on mobile is a battery and performance risk during a demo.
- **WebGL detection:** If `WebGL` is not available, show Plan B.
- Both plans wrapped in `<div role="img" aria-label="Interactive visualization showing Boot Force club network with X members, Y events, and Z projects">`

3.4 Stats Bar

- Four StatCounter components in a row
- Data fetched server-side:
    - Members: `COUNT(*) FROM profiles`
    - Events Hosted: `COUNT(*) FROM active_events WHERE status = 'finished'`
    - Challenges: `COUNT(*) FROM active_challenges`
    - Projects: `COUNT(*) FROM active_projects WHERE is_archived = false`
- Numbers animate from 0 to value when the section scrolls into view
- JetBrains Mono font for numbers
- ScrollReveal wrapper for entrance animation

3.5 Upcoming Events Strip

- Fetch: top 3 `active_events WHERE status = 'upcoming' AND is_published = true ORDER BY date_time ASC LIMIT 3`
- Horizontal scrollable row of event cards (ScrollArea component)
- Each card: title, category badge, date (formatted with date-fns), CountdownTimer component
- Click card → `/events/[id]`
- "View All Events →" link at the end

- If no upcoming events: "No upcoming events. Stay tuned!" EmptyState
- ScrollReveal and StaggerContainer for entrance

3.6 Featured Challenges Carousel

- Fetch: top 3 `active_challenges WHERE status = 'active' ORDER BY deadline ASC LIMIT 3`
- Auto-rotating carousel (5-second interval, pauses on hover)
- Each card: title, difficulty badge (color-coded), category badge, deadline text
- Click → `/challenges/[id]`
- "View All Challenges →" link
- Carousel controls: dots + arrow buttons (accessible with aria-labels)

3.7 Featured Projects Carousel

- Fetch: top 3 `active_projects WHERE is_featured = true ORDER BY created_at DESC LIMIT 3`
- Same carousel pattern as challenges
- Each card: thumbnail, title, tech stack TagList, likes count
- Click → `/projects/[id]`
- "View All Projects →" link

3.8 Top 5 Leaderboard Preview

- Fetch: `SELECT id, full_name, avatar_url, total_score FROM profiles ORDER BY total_score DESC LIMIT 5`
- Compact table: rank number, avatar, name, score, RankBadge component
- "View Full Leaderboard →" link
- This is NOT real-time on the homepage (uses ISR cached data). Real-time is only on `/leaderboard`.

3.9 Announcements Ticker

- Client Component
- Fetches active announcements on mount
- Supabase Realtime subscription on `announcements` table
- Horizontal marquee/scrolling text with announcement titles
- If no announcements: section hidden entirely
- Styled with subtle background strip

3.10 Footer — Full Implementation

- Already built in Phase 1, now populated with:
  - Club logo and "Boot Force" text
  - Quick links: all main navigation routes
  - Social media icons (placeholders URLs for now)
  - "Join Boot Force" CTA
  - Copyright text

3.11 Responsive Verification

- Test every section at: 375px (iPhone SE), 768px (iPad), 1280px (desktop)
- Hero adjusts text size, visualization switches to 2D on mobile
- Stats bar wraps to 2x2 grid on mobile
- Carousels are swipeable on touch
- All text readable, all buttons tappable (44x44px minimum)

What Can Be Automated (AI Agent)

- Homepage page component with all sections structured
- Server-side data fetching queries
- StatCounter, CountdownTimer component logic
- Carousel component (using a lightweight approach or custom with Framer Motion)
- Announcements ticker with Realtime subscription
- Footer content population
- Responsive layout classes

What Requires Manual Supervision

- Club Pulse visualization — this is the most visually subjective and technically complex component. The 2-hour decision point must be actively monitored.
- Hero section visual composition — gradient, text placement, CTA button positioning, how the visualization integrates with the hero. This is pure design judgment.
- Animation timing — do the scroll animations feel smooth? Is the stagger delay right? Does the carousel rotation speed feel natural?
- Overall visual impression — does the homepage look "cinematic"? This is a subjective call that requires human eyes.
- Mobile layout — does everything reflow gracefully? Any overflow? Any text too small?

Definition of Done

- Homepage hero section renders with tagline and CTA buttons
- Club Pulse visualization renders (3D or 2D, based on the decision point)
- Mobile fallback (2D) renders on screens < 768px
- Visualization is wrapped in accessible `role="img"` with `aria-label`
- Stats counters animate on scroll (counting up from 0 to seed data values)
- Upcoming events strip shows seed event cards with countdown timers
- Featured challenges carousel rotates and is interactive
- Featured projects carousel rotates and is interactive
- Top 5 leaderboard preview shows seed data
- Announcements ticker scrolls with seed announcement
- Navigation bar: responsive, auth-state-aware (Sign In vs. avatar dropdown)
- Mobile navigation works (hamburger → drawer with all links)
- Footer complete with all links
- Page loads in under 3 seconds on simulated 4G throttle (Chrome DevTools)

- Responsive on 375px, 768px, 1280px — no overflow, no unreadable text, no broken layouts
- All scroll-triggered animations fire correctly
- Page transition works when navigating from homepage to any other route
- Committed to GitHub with granular commits
- Live Vercel deployment updated and functional

PHASE 4: TEAM SECTION & USER PROFILES

Goal

Build the people pages — the curated team showcase, public user profiles, the personal dashboard, and account settings. These pages demonstrate the "club-specific" nature of the platform (15% of score) and provide the member experience backbone.

Scope

4.1 Team Page (/team)

- Page title with ScrollReveal animation
- Filter bar:
  - Role Category dropdown: All, Leads, Core Team, Members
  - Batch Year dropdown: populated from available years in seed data
  - Domain dropdown: populated from available domains in seed data
  - Alumni toggle switch
- Filters update the displayed cards. Filtering is client-side (small dataset, all loaded at once from server fetch)
- Hierarchical sections: "Club Leads" → "Core Team" → "Members"
- Each section header visible only if that category has cards to display
- Grid of team member cards per section
- StaggerContainer for entrance animation

4.2 Team Member Card

- Circular photo (Next.js <Image>, 120px, with fallback to initials avatar)
- Name (Space Grotesk 600)
- Designation
- Batch year badge
- Domain badge (color-coded by domain)
- Skill TagList (horizontal scrollable)
- GitHub and LinkedIn icon links (open in new tab)
- Hover: card lifts (scaleOnHover variant), shadow increases
- Click: opens Dialog/Modal with full detail view:
  - Larger photo
  - Full name and designation

    - All social links
    - Full skills list
    - If alumni: current role and company
    - Close button (X) and click-outside-to-close

4.3 Alumni Toggle

- When OFF: shows `team_members WHERE is_alumni = false`
- When ON: shows `team_members WHERE is_alumni = true`
- Toggle transition: cards fade out, new cards fade in (Framer Motion AnimatePresence)
- Alumni cards have a subtle "Alumni — Class of {batch_year}" badge

4.4 Public User Profile (`/profile/[id]`)

- Server Component fetches: profile, user_badges, submission stats
- Layout:
    - Left column (or top on mobile): avatar (large), full name, bio, tech stack tags, GitHub/LinkedIn links
    - Right column (or bottom on mobile):
        - RankBadge display (computed from total_score)
        - Stats row: total score, challenges attempted, events attended, current streak
        - Earned badges: grid of badge icons with names
        - If streak ≥ 7: flame animation next to streak number
- Responsive: stacks vertically on mobile

4.5 Personal Dashboard (`/dashboard`)

- Auth required (member route)
- Server Component fetches: own profile, own submissions with challenge titles, own event registrations with event titles, own badges
- Layout:
    - Welcome message: "Welcome back, {name}"
    - Stats cards: Total Score, Rank Badge, Current Streak, Challenges Completed, Events Attended
    - Recent submissions: list of last 5 submissions with challenge title, score (or "Pending"), submitted date
    - Upcoming registered events: list of events the user has registered for that haven't finished
    - Earned badges section
    - Activity heatmap placeholder (visual only in this phase — functional in Phase 8B)
    - Score over time chart placeholder (visual only — functional in Phase 8B)

4.6 Account Settings (`/profile/settings`)

- Auth required
- Pre-filled form with current profile data
- Sections:

    - **Profile Information:** avatar upload (with preview), full name, bio (textarea, max 500 chars), GitHub URL, LinkedIn URL, tech stack tags (input with add/remove)
    - **Password Change:** current password, new password, confirm new password. Zod validation.
    - **Notification Preferences:** three toggle switches (event reminders, challenge deadlines, announcements)
- Each section has its own "Save" button → separate Server Actions
- Success toast on save. Error toast on failure.

4.7 Admin Team Management

- `/admin/team` — list all team members with edit/remove buttons
- `/admin/team/new` — create form: name, photo upload, designation, role_category select, batch_year, domain, skills input, GitHub URL, LinkedIn URL, is_alumni toggle, alumni fields (conditional)
- `/admin/team/[id]/edit` — same form, pre-filled
- Remove: confirmation dialog → hard delete → revalidatePath('/team')
- Reorder: display_order number input per row (drag-and-drop is nice-to-have, manual number is sufficient)
- All admin actions write to audit_log

What Can Be Automated (AI Agent)

- Team page layout with filter bar and card grid
- Team member card component
- Modal detail view
- Profile page layout
- Dashboard layout with stat cards
- Settings form with Zod validation and React Hook Form
- Admin CRUD pages (forms, tables, actions)

What Requires Manual Supervision

- Card hover animations — do they feel tactile and polished?
- Modal transition — does it open/close smoothly?
- Filter interaction — does the AnimatePresence transition look good when switching between filtered views?
- Avatar upload flow — does the preview work? Does the file get uploaded to Supabase Storage correctly?
- Dashboard layout — is the information hierarchy clear? Is the most important info visible first?
- Mobile layout — do cards reflow into a single column? Is the modal usable on mobile (full-screen sheet instead of centered modal)?

Definition of Done

- Team page renders with hierarchical sections populated from seed data
- Team cards display all fields with hover animation

- Click card opens modal with full details
- Filter dropdowns work: selecting a filter updates the visible cards
- Alumni toggle works: switching shows/hides alumni with animation
- Public profile page renders for any user with correct data
- RankBadge displays correctly based on total_score
- Personal dashboard shows joined events, submissions, badges, stats
- Account settings: profile edit saves correctly to database
- Account settings: password change works
- Account settings: notification preferences toggle and save
- Avatar upload: select file → preview → save → stored in Supabase, profile updated
- Admin can add a new team member via form
- Admin can edit an existing team member
- Admin can remove a team member (with confirmation)
- All pages responsive on 375px, 768px, 1280px
- Committed and deployed

PHASE 5: EVENTS SYSTEM

Goal

Build the event discovery and participation flow — the most common interaction a club member will have with the platform. This directly serves the "functionality" (20%) and "club-specific modules" (15%) scoring categories.

Scope

5.1 Events Listing Page (/events)

- Page title
- Tab navigation: "Upcoming" | "Past" (shadcn/ui Tabs)
- Search bar: debounced input (300ms), updates URL search params (/events?q=workshop), Server Component reads param and filters by title ILIKE
- Category filter dropdown: All, Workshop, Hackathon, Talk, Competition, Social
- Grid of event cards

5.2 Event Cards

- Banner image (Next.js <Image>, 16:9 aspect ratio)
- Title
- Category badge (color-coded per category)
- Date and time (formatted: "Jan 15, 2025 • 3:00 PM")
- Venue/Mode badge (Online/Offline/Hybrid)
- Registration count ("42 registered")
- Upcoming events: CountdownTimer component

- Past events: no countdown, shows "Completed" badge
- Hover: card lifts, shadow increases
- Click → `/events/[id]`
- StaggerContainer for card entrance animation

5.3 Event Detail Page (`/events/[id]`)

- Banner image (full width, 16:9)
- Title (h1, Space Grotesk)
- Row of metadata: category badge, date, venue/mode, registration count
- Countdown timer (if upcoming, large format)
- Registration button:
  - Visitor: "Sign in to register" → `/login?redirect=/events/{id}`
  - Member (not registered): "Register" (red primary button) → Server Action → success toast → button changes to "Registered ✓" (green)
  - Member (registered): "Registered ✓" with "Cancel Registration" secondary link → Server Action → toast → button reverts to "Register"
  - Event full: "Registration Full" (disabled)
  - Event finished: "This event has ended" (disabled)
- Description section: rich text rendered with DOMPurify + Tailwind prose classes
- Schedule section: timeline visual — each slot with time, title, description
- Speakers section: row of speaker cards with photo, name, role
- FAQs section: Accordion component
- If finished and results available: Results section with winner highlights
- If gallery album linked: "View Event Gallery →" button → `/gallery/{albumId}`

5.4 Registration Flow (Server Action)

- Validate: user authenticated, event exists, event is upcoming, event not full, user not already registered
- Insert into `event_registrations`
- Trigger fires → `events.registration_count` increments
- Return success
- On client: optimistic update of button state, toast notification
- Cancellation: same flow in reverse, status set to 'cancelled', trigger decrements count

5.5 Admin Event Management

- `/admin/events` — table listing all events (including drafts and deleted)
  - Columns: title, category, date, status, registration count, is_published
  - Actions: edit, publish/unpublish, soft-delete, view registrations
  - Filter by status, category
- `/admin/events/new` — create event form:
  - Title input
  - Description: Tiptap rich text editor (dynamically loaded)

    - Banner image upload (with preview)
    - Date and time pickers (start and end)
    - Venue input
    - Mode select (online/offline/hybrid)
    - Category select
    - Max participants input (optional)
    - Schedule builder: add/remove time slots, each with time, title, description
    - Speakers builder: add/remove, each with name, role, image URL
    - FAQs builder: add/remove, each with question and answer
    - Results (for finished events): winners list, summary
    - Publish toggle
    - "Create Event" button → Server Action → audit log → redirect to admin events list → revalidatePath('/events')
- `/admin/events/[id]/edit` — same form, pre-filled
- Publish/Unpublish: toggle → Server Action → audit log → revalidatePath
- Soft delete: confirmation dialog → Server Action sets `is_deleted = true` → audit log → revalidatePath
- **Attendance management** (for finished events):
    - View registrations list for the event
    - Each registration row: user name, registration date, status
    - "Mark as Attended" checkbox per row
    - "Mark All as Attended" bulk button
    - Save → Server Action updates `event_registrations.status` → audit log

What Can Be Automated (AI Agent)

- Events listing page with tabs, search, filter
- Event card component
- Event detail page layout
- Registration Server Action
- Admin event form with all field types
- Schedule/speakers/FAQs builder components (dynamic add/remove field groups)
- Attendance management table
- Tiptap editor integration (dynamic import wrapper)

What Requires Manual Supervision

- Tiptap editor: verify it loads correctly with `ssr: false`, no hydration errors
- Rich text rendering: verify DOMPurify sanitizes correctly, prose styles look good
- Countdown timer: verify it ticks correctly, shows the right values, handles timezone correctly
- Registration flow: test the full cycle — register, verify count updates, cancel, verify count decrements
- Banner image upload: verify the full flow — select, preview, upload to Storage, URL saved to event
- Schedule/speakers/FAQs builder UX: are these intuitive to use? Easy to add/remove items?
- Mobile layout: event detail page with many sections — does it flow well vertically?

Definition of Done

- Events page renders with Upcoming/Past tabs
- Event cards show all fields including live countdown timer
- Search works: type a query, results update
- Category filter works
- Clicking a card navigates to event detail page
- Event detail page renders: banner, description (rich text), schedule, speakers, FAQs
- Registration button: works for authenticated member, shows "sign in" for visitor
- Registration button updates state after registering (optimistic update)
- Cancel registration works and reverts button state
- Registration count on card updates after registration
- Past events tab shows completed events with results
- Admin can create event with rich text editor, image upload, all fields
- Admin can edit event
- Admin can publish/unpublish event
- Admin can soft-delete event
- Admin can mark attendance on finished events
- New/edited events appear on public page after revalidation
- All pages responsive
- Committed and deployed

PHASE 6: CHALLENGES, SUBMISSIONS & REAL-TIME LEADERBOARD

Goal

Build the competitive engine — the feature that differentiates this from a static club website. The real-time leaderboard is the technical showpiece for the demo. This phase directly impacts "Innovation & Features" (20%) and "Functionality" (20%).

Scope

6.1 Challenge Listing Page (`/challenges`)

- Page title
- Filter bar: Difficulty (All/Easy/Medium/Hard), Category (All + 4 categories), Status (All/Active/Closed)
- Search bar: debounced, server-side ILIKE on title
- Grid of challenge cards

6.2 Challenge Card

- Title
- Difficulty badge: Easy = green (#22c55e), Medium = golden (#F5A623), Hard = red (#FC0204)

- Category badge
- Deadline: formatted date + "X days remaining" relative text (or "Closed" if past)
- Submission count (derived from seed data)
- Status dot: green pulse for active, grey for closed
- Hover animation
- Click → `/challenges/[id]`

6.3 Challenge Detail Page (`/challenges/[id]`)

- Title, difficulty badge, category badge, deadline with countdown
- Problem statement: rich text rendered with DOMPurify + prose
- Scoring rules (if provided): rendered from JSONB
- Tags displayed as TagList
- Submit button:
  - Visitor: "Sign in to submit" → `/login?redirect=/challenges/{id}/submit`
  - Member: "Submit Solution" → `/challenges/[id]/submit`
  - Challenge closed: "Submissions Closed" (disabled)
- **Per-challenge leaderboard:**
  - Table: rank, avatar, name, score, time_bonus, total, submitted_at
  - Realtime subscription on `submissions` table filtered by this challenge_id
  - Update pattern: setQueryData → cancelQueries → invalidateQueries
- **User's submission history** (if authenticated):
  - List of own past submissions for this challenge
  - Each: submitted_at, GitHub URL (clickable), live URL (clickable), status badge, score (if graded), feedback (if provided)

6.4 Submit Solution Page (`/challenges/[id]/submit`)

- Auth required
- Challenge title and deadline displayed for context
- Form:
  - GitHub Repository URL: text input with validation
    - Regex: must match `https://github.com/...` pattern
    - Error message if format invalid
  - Live Hosted URL: text input (optional) with URL validation
  - Duplicate check: before submit, query `submissions` for this user + this challenge + submitted in last 60 seconds. If found, show warning dialog "You submitted recently. Submit again?"
- Submit button → Server Action:
  - Validates URLs server-side (same Zod schema)
  - Inserts into `submissions` with status 'submitted'
  - Success toast: "Solution submitted successfully!"
  - Redirect to `/challenges/[id]` (the detail page, where they can see their submission in history)

6.5 Global Leaderboard Page (`/leaderboard`)

- Full page, Client Component
- **NOT server-rendered** — fetches on mount via TanStack Query
- Table: Rank, Avatar, Name, Total Score, Rank Badge, Challenges Completed
- Top 3 get special styling: gold (#F5A623), silver (#A0AEC0), bronze (#CD7F32) accent on their row
- Initial fetch: `profiles ORDER BY total_score DESC LIMIT 100`
- **Realtime subscription** on `profiles` table for `total_score` changes:
  - On UPDATE event: `cancelQueries → setQueryData (update score, re-sort) → invalidateQueries`
  - Full pattern as specified in Section 2.5
- Rank badges computed client-side from score thresholds
- "Challenges Completed" derived from count of graded submissions per user (fetched alongside leaderboard data via a join or separate query)

6.6 Admin Challenge Management

- `/admin/challenges` — table listing all challenges
  - Columns: title, difficulty, category, status, submissions count, deadline
  - Actions: edit, close, soft-delete
- `/admin/challenges/new` — create form:
  - Title, rich text editor for description and problem statement (two separate editors)
  - Difficulty select, category select
  - Start date picker, deadline picker
  - Max score input, max time bonus input
  - Scoring rules text area (JSON)
  - Tags input
  - "Create Challenge" button → Server Action → audit log → redirect → revalidatePath
- `/admin/challenges/[id]/edit` — same form, pre-filled
- Close challenge: sets status to 'closed' → Server Action → audit log → revalidatePath

6.7 Admin Submission Review (`/admin/submissions`)

- Filter dropdown: by challenge
- Table: user name, challenge title, submitted_at, GitHub URL (external link), live URL (external link), status, score
- Click row → inline expansion or side panel with:
  - Full submission details
  - Score input (0 to max_score)
  - Time bonus: auto-calculated display: `max(0, (deadline - submitted_at) / (deadline - start_date)) * max_time_bonus`, rounded to integer. Editable by admin.
  - Feedback textarea
  - Status select: submitted → under_review → graded
  - "Save Review" button → Server Action:
    - Updates submission row (score, time_bonus, feedback, status, reviewed_by, reviewed_at)
    - Trigger fires → recalculates `profiles.total_score`
    - Creates notification for the student

    - Writes to audit log
    - revalidatePath for relevant pages

6.8 Realtime Subscription Setup

- Custom hook: `useRealtimeLeaderboard()`
  - Creates Supabase channel subscribed to `profiles` table `UPDATE` events
  - On event: calls the cancelQueries → setQueryData → invalidateQueries pattern
  - Cleanup on unmount: `supabase.removeChannel()`
  - Connection status handling: toast on disconnect, invalidateQueries on reconnect
- Custom hook: `useRealtimeChallengeLeaderboard(challengeId)`
  - Same pattern but subscribed to `submissions` table filtered by `challenge_id`

What Can Be Automated (AI Agent)

- Challenge listing page with filters and cards
- Challenge detail page layout
- Submission form with Zod validation
- Global leaderboard table component
- TanStack Query setup for leaderboard
- Admin challenge CRUD pages
- Admin submission review panel
- Server Actions for submission and grading
- Realtime hook boilerplate

What Requires Manual Supervision

- **THE REAL-TIME DEMO:** This must be tested thoroughly. Open two browser tabs — grade in one, watch the leaderboard in the other. Verify the score updates, the rank re-sorts, and there's no visual glitch. This is the centerpiece of the demo.
- URL validation regex: test with various GitHub URL formats to ensure it doesn't reject valid URLs or accept invalid ones
- Time bonus calculation: verify the formula produces sensible values with the seed data
- Leaderboard top 3 styling: visual check that gold/silver/bronze accents look good
- Subscription cleanup: navigate away from leaderboard and back — verify no duplicate subscriptions or memory leaks
- Connection drop handling: simulate by toggling network in DevTools — verify toast appears and recovery works

Definition of Done

- Challenge listing page renders with filters (difficulty, category, status)
- Challenge cards display all fields with correct difficulty color coding
- Search works on challenge titles
- Challenge detail page renders problem statement (rich text)

- Submit solution form validates GitHub URL format
- Duplicate submission warning works (within 60 seconds)
- Submission creates a row in database with status 'submitted'
- User's submission history shows on challenge detail page
- Per-challenge leaderboard shows ranked submissions
- Global leaderboard page loads top 100 profiles by score
- **CRITICAL TEST: Open two tabs. Grade a submission in tab 1 (admin). Leaderboard in tab 2 updates in real-time WITHOUT page refresh.**
- Top 3 on leaderboard have gold/silver/bronze accent styling
- RankBadge displays correctly next to each leaderboard entry
- Admin can create challenges with all fields
- Admin can edit and close challenges
- Admin can view submissions, score them, add feedback
- Time bonus auto-calculates and displays to admin
- Grading a submission triggers a notification for the student
- All actions logged in audit_log
- All pages responsive
- Committed and deployed

PHASE 7: PROJECTS SHOWCASE & MEDIA GALLERY

Goal

Build the visual proof of the club's work and memories. The project showcase and gallery are high-impact "club-specific modules" (15%) with relatively lower technical complexity. This phase should be faster than the previous two.

Scope

7.1 Projects Listing Page (`/projects`)

- Page title
- Filter bar: Tech Stack (multi-select from available tags), Year, Domain
- Search bar: searches project titles
- Featured projects (where `is_featured = true`) pinned at top with a "Featured" badge (#FCF604 on dark card)
- Grid of project cards

7.2 Project Card

- Thumbnail (Next.js `<Image>`, 16:9)
- Title
- Description preview (first 100 characters + "...")
- Tech stack TagList

- Team member names
- Likes count (heart icon + number)
- Hover animation
- Click → `/projects/[id]`

7.3 Project Detail Page (`/projects/[id]`)

- Title, tech stack tags, team members, domain, year
- Screenshots carousel:
  - Swipeable on mobile (touch events)
  - Arrow buttons on desktop
  - Dots indicator showing current slide
  - Framer Motion slide transition
- Description: rich text with DOMPurify + prose
- Links row: GitHub, Live Demo, Documentation (each in a styled button/link, opens new tab)
- Like button:
  - Visitor: tooltip "Sign in to like"
  - Member (not liked): outline heart icon → click → Server Action inserts `project_likes` → trigger updates `likes_count` → optimistic update (filled heart + count increment)
  - Member (already liked): filled heart → click → Server Action deletes `project_likes` → trigger updates `likes_count` → optimistic update
  - TanStack Query mutation with optimistic update for instant feedback

7.4 Gallery Albums Page (`/gallery`)

- Page title
- Grid of album cards, sorted by event_date descending
- Album card:
  - Cover image (Next.js `<Image>`)
  - Album title
  - Event date (formatted)
  - Image count badge (e.g., "24 photos")
  - Hover animation
  - Click → `/gallery/[albumId]`

7.5 Album Detail Page (`/gallery/[albumId]`)

- Album title, event name, date, description
- Masonry grid of images:
  - CSS columns approach: `columns: 3` on desktop, `columns: 2` on tablet, `columns: 1` on mobile
  - All images use Next.js `<Image>` with `loading="lazy"`
  - Break-inside-avoid on image containers
- Click any image → Lightbox:
  - Full-screen overlay (dark background, z-50)
  - Current image centered, object-fit contain

    - Left/Right navigation: arrow buttons + keyboard ArrowLeft/ArrowRight
    - Close: X button + keyboard Escape
    - Swipe: touch left/right on mobile
    - Image counter: "3 of 24"
    - Caption below image
    - Focus trapped (Radix Dialog or custom trap)
    - Transition: Framer Motion scale and opacity

7.6 Admin Project Management

- `/admin/projects` — table of all projects with feature/archive/delete controls
- `/admin/projects/new` — create form:
    - Title, rich text editor for description
    - Thumbnail upload with preview
    - Tech stack tags input (add/remove)
    - Team member names input (add/remove)
    - Links: GitHub, Live, Documentation
    - Domain input, year input, team size input
    - Screenshots upload (multi-file, same bulk upload flow as gallery)
    - Is Featured toggle
    - "Create Project" → Server Action → audit log → revalidatePath('/projects')
- `/admin/projects/[id]/edit` — same form, pre-filled
- Feature/Unfeature: toggle → Server Action → audit log → revalidatePath
- Archive: toggle → Server Action
- Soft delete: confirmation → Server Action

7.7 Admin Gallery Management

- `/admin/gallery` — list all albums with create/edit/delete
- Create album: title, event date, description, event link (dropdown of events), cover image upload
- `/admin/gallery/[albumId]` — image management:
    - Grid of current images with delete button per image
    - Upload section:
        - File input accepting multiple files
        - Client-side validation: type (.jpg/.jpeg/.png/.webp), size (max 5MB each)
        - Upload flow: parallel `Promise.allSettled()` → progress indicator → batch insert → error handling as specified in Section 8.7
    - Reorder: display_order number input per image
    - Delete image: confirmation → delete from database AND Storage
    - Set as cover: click image → Server Action updates album `cover_image_url`

What Can Be Automated (AI Agent)

- Projects listing and detail page layouts
- Project card component

- Screenshots carousel component
- Like button with TanStack Query mutation
- Gallery albums page and album detail page
- Masonry grid layout CSS
- Lightbox component (using Radix Dialog as base)
- Admin CRUD pages for projects and gallery
- Bulk upload UI and logic

What Requires Manual Supervision

- Masonry grid layout: verify images of different aspect ratios arrange nicely without gaps
- Lightbox: test keyboard navigation, touch swipe, focus trapping, escape to close
- Lazy loading verification: scroll to gallery, check network tab — images below fold should NOT load until scrolled into view
- Bulk upload: test with 10+ images — verify progress indicator, error handling, and final state
- Screenshots carousel: swipe feel on mobile — is it responsive and smooth?
- Like optimistic update: click rapidly — does it toggle correctly without sending duplicate requests?
- Overall visual quality: do the project thumbnails and gallery images look good? Are aspect ratios preserved?

Definition of Done

- Projects page renders with filter bar and card grid
- Featured projects pinned at top with "Featured" badge
- Project cards show all fields with hover animation
- Project detail page shows screenshots carousel, rich text description, links
- Like button works: toggles, count updates optimistically, auth required
- Gallery page shows album cards with image counts
- Album detail shows masonry grid of images
- Lazy loading verified: below-fold images load on scroll (check network tab)
- Lightbox works: open, navigate left/right (arrows, keyboard, swipe), close (X, Escape)
- Lightbox shows image counter and caption
- Admin can create/edit/feature/archive/soft-delete projects
- Admin can create albums and upload images (single and bulk)
- Bulk upload shows progress and handles errors per file
- Admin can delete images (from database and Storage)
- Admin can set album cover image
- All pages responsive
- Committed and deployed

PHASE 8A: ADMIN PANEL, NOTIFICATIONS, BLOG

Goal

Complete the admin experience and add member-facing features: the analytics dashboard, user management, notifications system, and blog. These features round out the "Functionality" and "Innovation" scoring categories.

Scope

8A.1 Admin Analytics Dashboard (`/admin`)

- Welcome message: "Welcome, {admin_name}"
- Quick stats cards (server-fetched):
  - Total Users: `COUNT(*) FROM profiles`
  - Total Events: `COUNT(*) FROM active_events`
  - Total Challenges: `COUNT(*) FROM active_challenges`
  - Total Submissions: `COUNT(*) FROM submissions`
  - Active Users (7 days): `COUNT(*) FROM profiles WHERE last_active_date >= NOW() - INTERVAL '7 days'`
- Charts (Recharts, client-side):
  - New signups over time (line chart, last 30 days): `SELECT DATE(created_at), COUNT(*) FROM profiles GROUP BY DATE(created_at) WHERE created_at >= NOW() - INTERVAL '30 days'`
  - Submissions over time (line chart, last 30 days): same pattern on `submissions`
  - Event registrations over time (line chart, last 30 days): same pattern on `event_registrations`
- Recent admin actions (from audit_log, last 10): table with admin name, action, entity, timestamp

8A.2 User Management (`/admin/users`)

- Table: avatar, full name, email, role, total_score, created_at, is_banned
- Search by name or email (server-side ILIKE)
- Pagination: server-side, 20 per page
- Actions per user:
  - "Promote to Admin": confirmation dialog → Server Action (checks caller is admin, updates `profiles.role = 'admin'`) → audit log → toast
  - "Ban User": confirmation dialog → Server Action (sets `is_banned = true`) → audit log → toast
  - "Unban User": same flow, sets `is_banned = false`
  - "View Profile": link to `/profile/[id]`

8A.3 Announcement Management (`/admin/announcements`)

- List of announcements: title, content preview, is_active badge, created_at
- Create form: title input, content textarea
- Toggle active/inactive: Switch component → Server Action → audit log → triggers Realtime update to homepage ticker
- Delete: confirmation → hard delete → audit log

8A.4 Audit Log (`/admin/audit-log`)

- Table: admin name, action, entity type, entity ID, details, timestamp
- Filters: by action type dropdown, by date range
- Pagination: cursor-based (append-only table), 50 per page
- Read-only. No edit or delete actions.
- Server Component with pagination params from URL search params

8A.5 In-App Notifications

- Notification bell in navbar (already visually present from Phase 1/3):
  - Now functional with real data
  - Badge showing unread count
  - Click opens Popover component
- Notification Popover:
  - Fetches on open: last 20 notifications for current user + active announcements where `created_at > last_dismissed_announcements_at`
  - Each notification: icon (color-coded by type), title, message, relative time
  - Unread: white/light background. Read: grey background.
  - Click notification: mark as read (Server Action updates `is_read = true`), navigate to link
  - "Mark all as read" button: Server Action updates all unread for this user
  - "Dismiss announcements" button: Server Action updates `profiles.last_dismissed_announcements_at` to now()
- **Realtime subscription** on `notifications` table:
  - Custom hook: `useRealtimeNotifications(userId)`
  - On INSERT where `user_id = current user`: setQueryData to prepend notification, increment unread count
  - No invalidateQueries needed — notifications are append-only

8A.6 Blog System

- `/blog` — listing page:
  - Grid of blog post cards
  - Each card: cover image, title, author avatar + name, published_at, tags
  - Search: by title
  - Filter: by tag
  - Click → `/blog/[id]`
- `/blog/[id]` — detail page:
  - Cover image (full width)
  - Title (h1)
  - Author info: avatar, name (link to profile), published date
  - Tags
  - Content: rich text rendered with DOMPurify + prose
- `/blog/new` — create page (auth required, any member):
  - Title input
  - Tiptap editor (dynamically loaded) for content

    - Cover image upload
    - Tags input
    - Status toggle: Draft / Published
    - "Save" button → Server Action:
        - If status = 'published' and `published_at` is null, set `published_at = now()`
        - Insert or update blog post
        - revalidatePath('/blog')
    - Success toast
- `/blog/edit/[id]` — edit page:
    - Same form, pre-filled
    - Only author or admin can access (Server Component checks ownership/role before rendering)

What Can Be Automated (AI Agent)

- Admin dashboard layout with stats cards and chart placeholders
- Recharts chart components with data queries
- User management table with actions
- Announcement CRUD
- Audit log viewer with filters and pagination
- Notification bell and popover component
- Realtime notification hook
- Blog listing, detail, and creation pages
- Tiptap editor integration (reusing from events)

What Requires Manual Supervision

- Recharts: verify charts render correctly with seed data. Line charts need enough data points to look meaningful.
- User management: test promote and ban flows thoroughly. Verify that a banned user is actually blocked (test middleware).
- Notifications: test the full flow — grade a submission (from admin submissions), verify notification appears in the student's bell without page refresh.
- Realtime notification: same test as leaderboard real-time — open two tabs, trigger action in one, notification appears in other.
- Blog: verify Tiptap saves and loads content correctly (no data loss between save and render). Verify DOMPurify doesn't strip legitimate formatting.
- Mobile: notification popover — does it work well on small screens? Should it be full-screen sheet on mobile?

Definition of Done

- Admin dashboard shows correct stats and renders charts with data
- User management: list, search, promote, ban, unban all work
- Promoting a user to admin grants them admin access immediately

- Banning a user blocks their login
- Announcement creation shows in homepage ticker via Realtime
- Announcement toggle active/inactive works
- Audit log displays all admin actions with correct timestamps
- Audit log filters and pagination work
- Notification bell shows unread count
- Clicking bell opens popover with notifications and announcements
- Clicking a notification marks it read and navigates to link
- "Mark all as read" works
- "Dismiss announcements" works
- **Real-time notification test: Grade a submission → notification appears in student's bell within 2 seconds without page refresh**
- Blog listing shows published posts
- Blog detail renders rich text content correctly
- Blog creation: write, upload cover image, save as draft or published
- Blog edit: loads existing content, saves changes
- Only author or admin can edit a blog post
- All pages responsive
- Committed and deployed

PHASE 8B: BADGES, STREAKS, CERTIFICATES, FINAL POLISH

Goal

Build the remaining features, polish everything, prepare for the demo, and deploy the final version. This is the finish line.

Scope

8B.1 Badge System

- Seed badge definitions (from Section 8.10):
  - First Submission, First Win, Challenge Veteran (5), Event Regular (3), Legend (1000)
  - Each with icon (can use Lucide icons as SVGs stored in the badges table icon_url, or emoji

as placeholder)

- Badge award logic: implemented in relevant Server Actions
  - After a submission is created (status = 'submitted'):
    - Check: is this the user's first submission ever? → Award "First Submission"
  - After a submission is graded:
    - Check: is this the highest score for this challenge? → Award "First Win"
    - Check: does this user now have 5+ unique challenges with graded submissions? → Award "Challenge Veteran"

    - Check: is `profiles.total_score` now >= 1000? → Award "Legend"
  - After event attendance is marked:
    - Check: does this user now have 3+ events with status = 'attended'? → Award "Event Regular"
  - Each check:
    1. Query `user_badges` for this user + this badge
    2. If already awarded, skip
    3. If not awarded and criteria met, insert into `user_badges`
    4. Create a notification: "You earned the {badge_name} badge!"
  - This is simple if/else logic in the Server Action, NOT a separate rules engine
- Badge display:
  - Public profile (`/profile/[id]`): grid of earned badges with icon, name, earned_at date
  - Dashboard: same grid with earned dates
  - Leaderboard: user's highest-tier badge shown next to their name (Legend > Expert > Veteran > etc.)
  - Unearned badges: optionally shown as greyed-out silhouettes on dashboard (motivational "what you can earn next")

8B.2 Streak System

- Login streak logic (implemented in middleware or auth state change listener):
  - On each authenticated request where `last_active_date != today (UTC)`:
    1. Read `profiles.last_active_date`
    2. Get today's date in UTC: `new Date().toISOString().split('T')[0]`
    3. Get yesterday's date in UTC
    4. If `last_active_date === yesterday`: increment `current_streak` by 1
    5. If `last_active_date === today`: do nothing (already counted today)
    6. If `last_active_date` is older than yesterday OR null: reset `current_streak` to 1
    7. If `current_streak > longest_streak`: update `longest_streak = current_streak`
    8. Set `last_active_date = today`
    9. Single UPDATE query to profiles
  - Important: this runs in middleware on the server, NOT in the browser. Prevents manipulation.
  - Performance: one additional DB query per first-request-of-the-day. Negligible.
- Streak display on dashboard:
  - "Current Streak: X days" with a fire emoji/icon
  - "Longest Streak: Y days"
  - If current streak >= 7: animated flame icon (Framer Motion pulse animation)
  - If current streak >= 30: special "On Fire" visual treatment

8B.3 Activity Heatmap

- On dashboard: GitHub-style contribution grid
- Data source: derived query (NO separate table, as decided):

```sql
SELECT activity_date, SUM(action_count) as total_actions FROM (
```

```
SELECT DATE(submitted_at) as activity_date, COUNT(*) as action_count
FROM submissions WHERE user_id = \$1 GROUP BY DATE(submitted_at)
UNION ALL
SELECT DATE(registered_at), COUNT(*)
FROM event_registrations WHERE user_id = \$1 AND status != 'cancelled' GROUP BY DATE(registered_at)
UNION ALL
SELECT DATE(created_at), COUNT(*)
FROM project_likes WHERE user_id = \$1 GROUP BY DATE(created_at)
UNION ALL
SELECT DATE(published_at), COUNT(*)
FROM blog_posts WHERE author_id = \$1 AND status = 'published' AND published_at IS NOT NULL GROUP BY DATE(published_at)
) as activity
GROUP BY activity_date
ORDER BY activity_date DESC
LIMIT 365
```

- Server Component fetches this data and passes to a Client Component for rendering
- Heatmap component:
  - Grid of small squares, 52 columns (weeks) × 7 rows (days)
  - Each square colored by intensity:
    - 0 actions: #1E5A8A (dark, blends with background)
    - 1 action: #3B82F6 (light blue)
    - 2-3 actions: #2563EB (medium blue)
    - 4+ actions: #FC0204 (bright red, brand color)
  - Hover on a square: tooltip showing date and action count
  - Month labels along the top
  - Day labels (Mon, Wed, Fri) along the left
  - Responsive: on mobile, show last 26 weeks (half year) instead of 52

8B.4 Score Over Time Chart

- On dashboard: line chart (Recharts) showing cumulative score after each graded submission
- Data: `SELECT submitted_at, score, time_bonus FROM submissions WHERE user_id = \$1 AND status = 'graded' ORDER BY submitted_at ASC`

- Transform to cumulative: each point is the running total of (score + time_bonus) up to that submission
- X-axis: dates. Y-axis: cumulative score.
- If no graded submissions: show empty state "Complete challenges to see your progress"

8B.5 Certificate Generation

- On-demand, client-side, single certificate at a time
- "Download Certificate" button appears:
  - On event detail page: if event is finished AND user's registration status = 'attended'
  - On dashboard: list of events where user attended, each with a "Download Certificate" button
- Button click flow:
  1. Determine certificate type: if user is in event's `results.winners` array → "Certificate of Achievement", else → "Certificate of Participation"
  2. Render a hidden HTML `<div>` with the certificate template (positioned off-screen or with opacity 0):
     - Dimensions: A4 landscape ratio (297mm × 210mm, rendered at 2x for quality: 2339px × 1654px)
     - Background: Deep Navy (#0B3A63)
     - Border: 2px Golden Orange (#F5A623), 32px margin from edge, rounded-xl
     - Top center: Boot Force logo (white SVG or PNG)
     - Main heading: "CERTIFICATE OF PARTICIPATION" or "CERTIFICATE OF ACHIEVEMENT" — Space Grotesk 700, #FFFFFF, 48px
     - "This is proudly awarded to" — Inter 400, #D9E2EC, 20px
     - Recipient name: user's full_name — Space Grotesk 700, #F5A623, 56px
     - "for participating in" — Inter 400, #FFFFFF, 20px
     - Event title — Space Grotesk 600, #FFFFFF, 32px
     - Event date — Inter 400, #D9E2EC, 18px
     - Certificate ID: bottom right — JetBrains Mono 400, #A0AEC0, 12px
       - ID format: `BF-{eventId.slice(0,8)}-{userId.slice(0,8)}` (deterministic, always same for same user+event)
     - "Boot Force Team" signature line: bottom center, line above text
  3. Call `html2canvas(hiddenDiv, { scale: 2, useCORS: true })` to render to canvas
  4. Call `jsPDF` to create PDF from canvas:
     - Page size: A4 landscape
     - Add canvas as image
     - Save as `BootForce_Certificate_{eventTitle}_{userName}.pdf`
  5. Insert/update `certificates` row: set `downloaded_at = now()`
  6. Remove the hidden div from DOM
  7. Total time: 1-2 seconds
- No server-side generation. No bulk generation. One certificate, one click, instant download.

8B.6 Alumni Section Enhancement

- Already built in Phase 4 as a toggle on the team page

- Enhancement: ensure alumni cards display `alumni_current_role` and `alumni_current_company` prominently
- If seed data doesn't have alumni, add 1-2 alumni entries to the seed script
- Visual: subtle "Alumni — Class of {batch_year}" badge, slightly different card background tint

8B.7 OAuth Activation (If Time Allows)

- Google OAuth:
  1. Create OAuth app in Google Cloud Console
  2. Set authorized redirect URI: `https://{your-url}.vercel.app/auth/callback`
  3. Copy Client ID and Client Secret
  4. Add to Supabase Dashboard → Auth → Providers → Google
  5. Add `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` to Vercel environment variables
  6. Enable the Google button on login/signup pages (remove disabled state and tooltip)
- GitHub OAuth:
  1. Create OAuth app in GitHub Developer Settings
  2. Set callback URL: `https://{your-url}.vercel.app/auth/callback`
  3. Copy Client ID and Client Secret
  4. Add to Supabase Dashboard → Auth → Providers → GitHub
  5. Add to Vercel environment variables
  6. Enable the GitHub button
- Test: sign up with Google → profile created → login works → role is 'member'
- Test: sign up with GitHub → same flow
- If this takes more than 1 hour, abandon and keep buttons disabled. It's worth 0 points if it doesn't work.

8B.8 SEO & Meta Tags

- Root layout `metadata` object:
  - Title: "Boot Force — Technical Club Hub | CMR University"
  - Description: "The official digital hub for Boot Force, CMR University's premier technical club. Events, challenges, projects, and more."
  - Open Graph: title, description, image (a designed OG image or the hero screenshot), URL
  - Twitter card: same data
- Per-page metadata using `generateMetadata` function:
  - `/events/[id]`: title = event title, description = first 160 chars of event description (stripped of HTML)
  - `/challenges/[id]`: title = challenge title
  - `/projects/[id]`: title = project title
  - `/blog/[id]`: title = post title, image = cover image
  - `/profile/[id]`: title = user's full name
- Verify: paste a page URL into a social media link preview tool → correct title, description, and image appear

8B.9 Performance Optimization

- Images: verify ALL images use Next.js `<Image>` component with:
  - `priority` on above-the-fold images (hero banner, first visible cards)
  - `loading="lazy"` on below-fold images (gallery, lower cards)
  - `sizes` prop set correctly for responsive images
  - Automatic WebP conversion (Next.js does this by default)
- Code splitting: verify Tiptap, Three.js/R3F, Recharts are dynamically imported and NOT in the main bundle
  - Check: load homepage → open Network tab → Tiptap/Three.js/Recharts bundles should NOT be loaded
  - These should only load on pages that use them
- Bundle analysis: run `npm run build` and check output for unexpectedly large chunks
- Lighthouse audit on homepage:
  - Target: Performance > 80, Accessibility > 90
  - Fix any critical issues flagged

8B.10 Final Responsive Audit

- Test EVERY page at three breakpoints: 375px (iPhone SE), 768px (iPad), 1280px (desktop)
- Checklist per page:
  - No horizontal overflow / no horizontal scroll on the body
  - All text readable without zooming
  - All buttons/links tappable (44x44px minimum touch target)
  - Images don't overflow containers
  - Modals are usable (full-screen sheet on mobile if needed)
  - Forms are usable (inputs aren't cut off, keyboard doesn't cover form)
  - Navigation works (hamburger menu, drawer)
  - Cards reflow correctly (single column on mobile, multi-column on desktop)

Pages to test:

1. Homepage
2. Team
3. Events listing
4. Event detail
5. Challenges listing
6. Challenge detail
7. Submit solution
8. Projects listing
9. Project detail
10. Gallery albums
11. Album detail + lightbox
12. Leaderboard
13. Blog listing
14. Blog detail

15. Blog create/edit
16. Public profile
17. Dashboard
18. Account settings
19. Login / Signup
20. Admin dashboard
21. Admin events (list + create/edit)
22. Admin challenges (list + create/edit)
23. Admin submissions
24. Admin team (list + create/edit)
25. Admin projects (list + create/edit)
26. Admin gallery (albums + images)
27. Admin users
28. Admin announcements
29. Admin audit log
30. 404 page
31. Error page

8B.11 README

- Comprehensive README.md in the GitHub repo:
  - Project title and one-line description
  - Screenshot of the homepage
  - Tech stack list with versions
  - Features list (bullet points)
  - Getting started:
    1. Clone repo
    2. `npm install`
    3. Create Supabase project
    4. Run SQL setup (link to SQL file in repo)
    5. Create `.env.local` with required variables (list them)
    6. `npm run dev`
    7. Run seed script: `npx tsx scripts/seed.ts`
  - Environment variables documentation (what each one is, where to get it)
  - Project structure overview
  - Deployment instructions (Vercel)
  - Credits / team members
  - License

8B.12 Demo Preparation

- Run seed script one final time for clean data
- Delete any garbage test content (blog posts, submissions from development)
- Verify admin login works: `admin@bootforce.com`

- Verify student pre-seeded data exists:
  - Student has submissions (some graded, one pending for demo)
  - Student has event registrations
  - Student has earned at least one badge
  - Student has some streak data
- Walk through the entire demo script (Section 12) once privately:
  - Tab 1: demo walkthrough
  - Tab 2: leaderboard
  - Grade a submission → leaderboard updates
  - Create an event → appears on public page
- Time the walkthrough — must be under 3 minutes
- Prepare mobile device or DevTools responsive mode for the responsive showcase
- Have backup: if WiFi is unreliable during demo, pre-record a short video of the real-time update moment as proof

What Can Be Automated (AI Agent)

- Badge award logic inserted into existing Server Actions (if/else checks)
- Streak calculation logic in middleware
- Activity heatmap component (grid rendering from data array)
- Score over time chart component
- Certificate HTML template component
- html2canvas + jsPDF download flow
- SEO metadata configuration
- README template generation

What Requires Manual Supervision

- Badge logic: manually test each badge trigger. Submit first solution → check "First Submission" badge awarded. Grade it as top score → check "First Win" awarded. Verify notifications created.
- Streak: log in → check streak increments. Wait a day (or manipulate `last_active_date` in database) → verify reset logic.
- Certificate visual: download a certificate → open the PDF → verify it looks professional. Text alignment, font rendering, border quality, logo clarity. html2canvas can have subtle rendering differences from browser display.
- Heatmap visual: verify the grid renders correctly with seed data. Check that colors map to the right intensity levels. Check mobile truncation (26 weeks).
- OAuth (if attempted): full flow test — signup with Google, verify profile created, login again, verify session works.
- Performance audit: run Lighthouse, read the report, fix critical issues.
- Responsive audit: manually check every page at every breakpoint. This cannot be automated — it requires human visual judgment.
- Demo rehearsal: run through the entire script. Note any rough transitions or moments of confusion. Adjust the script or fix issues.

Definition of Done

- Badge definitions seeded in database (5 badges)
- Submitting first challenge solution awards "First Submission" badge and creates notification
- Grading a submission as top score awards "First Win" badge
- Completing 5 unique challenges awards "Challenge Veteran" badge
- Attending 3 events awards "Event Regular" badge
- Reaching 1000 score awards "Legend" badge
- Badges display correctly on public profile and dashboard
- Login streak calculation works: increment on consecutive days, reset on gap
- Streak displays on dashboard with fire icon if >= 7
- Activity heatmap renders on dashboard with real derived data
- Score over time chart renders on dashboard with graded submissions
- "Download Certificate" button appears for attended finished events
- Clicking download generates and downloads a PDF certificate
- Certificate PDF looks professional: correct layout, text, colors, logo
- Certificate ID is deterministic (same user + same event = same ID always)
- Alumni cards show current role and company on team page
- OAuth buttons work (if activated) — or remain disabled with tooltip (if not)
- Every public page has correct Open Graph meta tags
- Link preview shows correct title, description, and image when URL is shared
- Lighthouse Performance score > 80 on homepage
- Lighthouse Accessibility score > 90 on homepage
- Tiptap, Three.js/R3F, and Recharts are NOT in the main bundle (verified via Network tab)
- All 31 pages pass responsive check at 375px, 768px, 1280px
- No horizontal overflow on any page at any breakpoint
- README is complete with setup instructions, tech stack, features, and environment variables
- Seed script produces clean, realistic demo data
- Demo script rehearsed end-to-end in under 3 minutes
- Real-time leaderboard update works during demo rehearsal
- No console errors on any page
- No broken links on any page
- Live Vercel deployment is the final version
- All commits pushed to GitHub with meaningful messages

SECTION 14: RISK REGISTER & MITIGATION

#	Risk	Likelihood	Impact	Mitigation
1	Three.js visualization looks bad or performs poorly	Medium	High (it's the innovation hook)	2-hour decision point in Phase 3. Plan B (2D SVG) is the fallback and takes ~2 hours to build. Demo script works with either version.
2	Supabase free tier limits reached	Low	High	1GB storage, 500MB database, 2GB bandwidth. With placeholder images and compressed photos, we stay well under. Monitor usage in Supabase dashboard.
3	Real-time leaderboard doesn't update during demo	Medium	Critical (it's the demo's centerpiece)	Test repeatedly before demo. Have a pre-recorded video of the update as backup. Verify WebSocket connection is stable on the demo network.

4	Tiptap rich text editor causes hydration errors	Medium	Medium	Dynamic import with `ssr: false`. Test in development. If issues persist, fall back to a plain textarea for the hackathon and render markdown with a library.
5	html2canvas renders certificate incorrectly	Medium	Low (certificates are Phase 8B)	Test certificate output early in Phase 8B. If rendering is broken, simplify the certificate template to basic elements that html2canvas handles reliably.
6	Auth middleware causes redirect loops	Low	High	Test all route combinations in Phase 2: public routes (no redirect), member routes (redirect to login), admin routes (redirect to home for non-admins). Edge cases: logged in user visiting

				/login (redirect to /dashboard).
7	Time runs out before all phases complete	Medium	Medium	Phases ordered by scoring impact. Phases 1-6 cover 80% of the scoring. Phases 7-8B are progressively less critical. Even completing through Phase 6 gives a competitive submission.
8	Demo WiFi is unreliable	Medium	High	Test real-time features on mobile hotspot as backup. Pre-record key moments (leaderboard update) as video backup. Preload all demo pages in browser cache before presenting.
9	Seed data looks fake or unconvincing	Low	Medium	Use realistic names, descriptions, and dates. Use randomuser.m

				e for professional-looking photos. Review seed data before final demo and replace anything that looks obviously placeholder.
10	Database triggers don't fire correctly	Low	High	Test every trigger explicitly in Phase 2: insert registration → verify count, grade submission → verify total_score. If triggers fail, fall back to application-level updates in Server Actions (less elegant but functional).
11	Supabase Realtime connection drops during demo	Low	High	Supabase has built-in reconnection. Toast appears on drop. On reconnect, invalidateQueries catches missed updates. Demo backup: if real-

				time fails, manually refresh the page to show the updated leaderboard.
12	OAuth configuration takes too long	High	Low (deferred to 8B)	Strict 1-hour timebox. If not working in 1 hour, abandon and keep buttons disabled. Worth 0 points if broken.
13	Mobile layout breaks on specific pages	Medium	Medium	Final responsive audit in Phase 8B covers all 31 pages. Use browser DevTools device emulation. Fix critical breaks immediately, accept minor imperfections on rarely-visited admin pages.
14	Vercel deployment fails	Low	Medium	Vercel auto-deploys on push. If build fails, check build logs immediately. Most common issue: TypeScript

				errors that pass in dev but fail in build (strict mode). Run `npm run build` locally before critical pushes.
15	Browser tab crashes due to Three.js memory leak	Medium	Medium	If using Plan A: keep polygon count low, use instanced meshes, dispose geometries and materials on unmount. If any instability, pivot to Plan B. Monitor browser memory usage during Phase 3.

SECTION 15: POST-HACKATHON ROADMAP

These features are explicitly out of scope for the hackathon but documented for future development:

Near-Term (1-2 months):

- Multi-club support: the platform spawning architecture originally envisioned
- Email notifications via Resend or SendGrid
- Automated challenge status (cron job to close challenges at deadline)
- Blog moderation: optional admin approval workflow
- Individual announcement dismissal (junction table)
- Full-text search with PostgreSQL `tsvector` and `pg_trgm`
- Automated testing suite (Playwright for E2E, Vitest for unit)
- Sentry for error monitoring

Medium-Term (3-6 months):

- Custom domain per club
- Event ticketing and paid workshops (Stripe integration)
- Mentorship matching system
- Discord/Slack integration for notifications
- Resume builder using profile data
- Comment system on blog posts and projects
- Admin roles with permissions (super admin, content admin, event admin)
- Database migration system with version control

Long-Term (6-12 months):

- Mobile app (React Native, sharing Supabase backend)
- PWA with offline capability
- Auto-grading challenge submissions via sandboxed code execution
- AI-powered project recommendations based on user skill tags
- Live collaborative event chat rooms
- Video hosting for event recordings
- API for third-party integrations
- Analytics export and reporting tools

APPENDIX A: SEARCH ARCHITECTURE

Page	Searchable Columns	Method	UI Pattern
Events	`title`	Server-side `ILIKE '%query%'`	Search input above filter bar. Debounced 300ms. Updates URL search params. Server Component reads param.
Challenges	`title`	Same	Same
Projects	`title`, `tech_stack` (array `@>`)	Same for title. Array contains for tech stack.	Same
Blog	`title`	Same	Same
Team	`name`, `designation`	Client-side filter (small dataset, all loaded)	Search input above filter bar. Instant filter.
Admin users	`full_name`, `email`	Server-side `ILIKE`	Same as events

Rich text description columns are NOT searched — ILIKE on HTML content matches HTML tags, not just text content. Title-only search is sufficient for the hackathon.

APPENDIX B: PAGINATION STRATEGY

Page	Method	Page Size	Implementation
Events listing	Server-side offset	12	URL param `?page=1`. Supabase `.range(from, to)`. Previous/Next buttons.
Challenges listing	Server-side offset	12	Same
Projects listing	Client-side load more	12 initial, 12 per load	"Load More" button. TanStack Query infinite query.
Gallery images (in album)	Infinite scroll	20 per batch	Intersection Observer triggers next batch. Cursor-based: `.lt('display_order', lastLoaded)`.
Blog listing	Server-side offset	10	Same as events
Leaderboard	Server-side, single load	100	Top 100 loaded once. No pagination. Sufficient for hackathon scale.
Admin submissions	Server-side offset	20	Same as events. With challenge filter.
Admin users	Server-side offset	20	Same
Admin audit log	Cursor-based	50	`.lt('created_at', lastTimestamp)`. "Load Older" button. Append-only table.

APPENDIX C: FILE UPLOAD SPECIFICATIONS

Client-Side Validation (before upload):

Check	Rule	Error Message
File type	`.jpg`, `.jpeg`, `.png`, `.webp` — check both MIME type and file extension	"Only JPG, PNG, and WebP images are allowed"
File size (avatars)	Max 2MB	"Avatar image must be under 2MB"
File size (all others)	Max 5MB	"Image must be under 5MB"

Upload Flow (single file):

1. Client validates file type and size
2. Client calls Server Action with the file
3. Server Action uploads to Supabase Storage with a unique path: `{bucket}/{entityId}/{timestamp}_{filename}`
4. If upload succeeds, server updates the database row with the new public URL
5. If the entity previously had an image (replacement), server deletes the old file from Storage
6. If old file deletion fails, log the orphan path to audit_log (non-blocking)
7. Return success with the new URL

Upload Flow (bulk — gallery):

1. Client validates each file individually (type, size)
2. Invalid files are rejected with per-file error messages before upload begins
3. Valid files are uploaded in parallel via `Promise.allSettled()`
4. Progress: track fulfilled count vs total. Display "Uploaded 7 of 12"
5. After all settle: collect fulfilled URLs, report rejected files with reasons
6. Batch insert fulfilled URLs into `gallery_images` via single `.insert([...])`
7. If batch insert fails: retry once
8. If retry fails: delete all successfully uploaded files from Storage, show error "Upload failed. Please try again."
9. Result: either all succeed (files in Storage AND database) or all are cleaned up (nothing orphaned)

Storage Path Convention:

Bucket	Path Pattern	Example
avatars	`{userId}/avatar.{ext}`	`abc123/avatar.webp`
banners	`events/{eventId}/{timestamp}.{ext}`	`events/def456/1705312345.jpg`
gallery	`albums/{albumId}/{timestamp}_{index}.{ext}`	`albums/ghi789/1705312345_03.jpg`
projects	`{projectId}/thumbnail.{ext}` or `{projectId}/screenshots/{index}.{ext}`	`jkl012/thumbnail.jpg`

APPENDIX D: ENVIRONMENT VARIABLES REFERENCE

Variable	Prefix	Exposure	Source	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	`NEXT_PUBLIC_`	Browser + Server	Supabase Dashboard → Settings → API → Project URL	Supabase client initialization
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	`NEXT_PUBLIC_`	Browser + Server	Supabase Dashboard → Settings → API → anon/public key	Client-side queries (RLS enforced)
`SUPABASE_SERVICE_ROLE_KEY`	None	Server only	Supabase Dashboard → Settings → API → service_role key	Server Actions that bypass RLS (badge awards, notifications)
`GOOGLE_CLIENT_ID`	None	Server only	Google Cloud Console → OAuth 2.0 Credentials	OAuth provider (Phase 8B)
`GOOGLE_CLIENT_SECRET`	None	Server only	Same	OAuth provider (Phase 8B)
`GITHUB_CLIENT_ID`	None	Server only	GitHub Developer Settings → OAuth Apps	OAuth provider (Phase 8B)
`GITHUB_CLIENT_SECRET`	None	Server only	Same	OAuth provider (Phase 8B)

Rules:

- Variables prefixed with `NEXT_PUBLIC_` are safe for the browser because RLS protects the database

- `SUPABASE_SERVICE_ROLE_KEY` bypasses ALL RLS — it must NEVER be in a file or variable prefixed with `NEXT_PUBLIC_`
- OAuth variables are only needed in Phase 8B. Until then, they're absent and the OAuth buttons remain disabled.
- All variables are set in both `.env.local` (local development) and Vercel Dashboard (production)
- `.env.local` is in `.gitignore` — never committed to the repository

APPENDIX E: COMPLETE ADMIN AUDIT LOG ACTIONS

Every admin Server Action writes to `admin_audit_log` before returning. The following is the complete list of logged actions:

Action String	Entity Type	When
`created_event`	event	Admin creates a new event
`updated_event`	event	Admin edits an event
`published_event`	event	Admin publishes an event
`unpublished_event`	event	Admin unpublishes an event
`deleted_event`	event	Admin soft-deletes an event
`marked_attendance`	event	Admin marks attendees for an event
`created_challenge`	challenge	Admin creates a challenge
`updated_challenge`	challenge	Admin edits a challenge
`closed_challenge`	challenge	Admin closes a challenge
`deleted_challenge`	challenge	Admin soft-deletes a challenge
`graded_submission`	submission	Admin grades a submission
`created_team_member`	team_member	Admin adds a team member
`updated_team_member`	team_member	Admin edits a team member
`deleted_team_member`	team_member	Admin removes a team member
`created_project`	project	Admin adds a project
`updated_project`	project	Admin edits a project

`featured_project`	project	Admin features/unfeatures a project
`archived_project`	project	Admin archives a project
`deleted_project`	project	Admin soft-deletes a project
`created_album`	gallery_album	Admin creates a gallery album
`updated_album`	gallery_album	Admin edits album details
`deleted_album`	gallery_album	Admin deletes an album
`uploaded_images`	gallery_image	Admin uploads images to an album
`deleted_image`	gallery_image	Admin deletes a gallery image
`promoted_user`	profile	Admin promotes a member to admin
`banned_user`	profile	Admin bans a user
`unbanned_user`	profile	Admin unbans a user
`created_announcement`	announcement	Admin creates an announcement
`toggled_announcement`	announcement	Admin activates/deactivates an announcement
`deleted_announcement`	announcement	Admin deletes an announcement
`created_badge`	badge	Admin creates a badge definition

Each log entry includes: `admin_user_id`, `action`, `entity_type`, `entity_id` (UUID of the affected entity), `details` (JSONB with relevant context like old/new values), `created_at`.

APPENDIX F: RANK BADGE THRESHOLDS

Computed in the application layer. NOT stored in the database.

Badge	Min Score	Max Score	Display Color	Used On
Newbie	0	99	Cool Grey (#A0AEC0)	Profile, dashboard, leaderboard
Contributor	100	499	Steel Blue (#1E5A8A)	Same
Expert	500	999	Golden Orange (#F5A623)	Same
Legend	1000	∞	Bright Red (#FC0204)	Same

Implementation: a utility function `getRankBadge(totalScore: number): { name: string, color: string }`

Used in: `RankBadge` component (built in Phase 1), leaderboard table, public profile, dashboard.

APPENDIX G: NOTIFICATION QUERY PATTERN

The notification bell combines two data sources into one list:

Source 1 — User Notifications:

```sql
SELECT id, title, message, type, is_read, link, created_at, 'notification' as source
FROM notifications
WHERE user_id = \$1
ORDER BY created_at DESC
```

```
LIMIT 20
```

Source 2 — Active Announcements (not dismissed):

```sql
SELECT id, title, content as message, 'announcement' as type, false as is_read, null as link, created_at, 'announcement' as source
FROM announcements
WHERE is_active = true
AND created_at > (SELECT last_dismissed_announcements_at FROM profiles WHERE id = \$1)
ORDER BY created_at DESC
```

Combined: Both queries run, results merged on the client, sorted by `created_at` descending. Announcements appear interleaved with notifications in chronological order.

Marking as read:

- Notifications: `UPDATE notifications SET is_read = true WHERE id = \$1 AND user_id = \$2`
- Announcements: "Dismiss all" updates `profiles.last_dismissed_announcements_at = now()`

Unread count: Count of notifications where `is_read = false` PLUS count of undismissed active announcements.

END OF BUILD BIBLE

This document is the single source of truth for the entire Boot Force project. Every architectural decision, every data model, every feature specification, every phase scope, and every risk mitigation is documented here. Any prompt given to an AI agent or any instruction given to a collaborator should reference this document.

Section 8.10 — ClubPulse Visualization Node Colors

Old: Members = `#FC0204`, Events = `#F5A623`, Challenges = `#FCF604`, Projects = `#1E5A8A`, Edges = `#D9E2EC`
New: Members = `#E6007E` (Hologram), Events = `#FFB000` (Warning), Challenges = `#00F0FF` (Glitch), Projects = `#00FF88` (Success), Edges = `#2D1B4E` (Glass Border) at 40% opacity

Phase 1 (Section 13) — Tailwind Config Colors

Old: Lists all CMR brand color names (navy, steel, blue-grey, brand-red, etc.)
New: Replace with:

```text
```

`void: '#0A0514'`

`glass: '#140A29'`

`glass-border: '#2D1B4E'`

`hologram: '#E6007E'`

`glitch: '#00F0FF'`

`light: '#F2F0FF'`

`shadow: '#A39BBA'`

`success: '#00FF88'`

`warning: '#FFB000'`

`error: '#FF3366'`

Also add to Phase 1 Tailwind config:

- `addVariablesForColors` plugin (from Aurora component)
- `aurora` keyframe animation (60s linear infinite)
- `spotlight` keyframe animation (2s ease 0.75s 1 forwards)

Phase 1 (Section 13) — shadcn Component Styling References

Old: References to navy backgrounds, red buttons, blue-grey borders
New: All references should use the Blade Runner token names (void, glass, hologram, glitch, etc.)

Phase 3 (Section 13) — Homepage Hero

Old: "Deep Navy (#0B3A63) background with subtle gradient overlay"
New: "Void (#0A0514) background with AuroraBackground component and Aceternity Spotlight beam"

Phase 3 — Navbar

Old: "semi-transparent on homepage hero (glassmorphism effect)"
New: "Glass card style: `bg-[#140A29]/40 backdrop-blur-md border-b border-[#2D1B4E]`"

Appendix F — Rank Badge Thresholds

Replace the color column with the new values listed above.

use new colours and design